Marine Race Arena: A Configurable HoloOcean Benchmark for Underwater Gate Racing and Team-Level Fleet Evaluation

Andrea Bedei[a], Lorenzo Bacchiani[a], Giovanni Pau[a], Roberto Girau[a]

[a]*Department of Computer Science and Engineering, University of Bologna, Italy*

Abstract

We present Marine Race Arena, a configurable and reproducible framework for developing and evaluating autonomous systems in underwater gate-racing tasks. Complete race scenarios can be specified through declarative configurations, including tracks, vehicles, onboard sensors, acoustic beacons, environmental currents, obstacles, starting conditions, race rules and penalties. A central design principle is the strict separation between autonomy and evaluation: controllers operate exclusively from onboard observations, acoustic measurements and optional inter-vehicle communication, while an independent referee accesses privileged simulator state only to validate ordered gate crossings, enforce the rules and compute the official results. This common interface enables different autonomy strategies to be evaluated under the same sensing and scoring conditions and supports both single-vehicle trials and cooperative multi-vehicle scenarios. The reference implementation includes onboard visual-acoustic gate perception, two interpretable rule-based controllers, leader-follower coordination mechanisms and reinforcement-learning controllers integrated through the same observation and action interface. Three ready-to-use race circuits with different geometric characteristics are provided together with structured logging and automated validation tools for reproducible experimentation. Experiments conducted in HoloOcean with a BlueROV2-class vehicle assess course completion, gate progression, timing, collisions, robustness to environmental currents and fleet coordination. The results show that Marine Race Arena can expose meaningful differences between autonomy strategies and operating conditions while maintaining a common evaluation protocol, providing a unified testbed for studying underwater perception, navigation, control, learning and cooperation under repeatable and progressively challenging racing scenarios.

Keywords: underwater robotics, autonomous racing, robotics benchmark, reproducible evaluation, onboard perception, multi-robot systems, reinforcement learning, HoloOcean

1. Introduction

Autonomous racing has emerged as an effective research format for evaluating perception, planning, control and real-time decision-making within a single quantitative task under repeatable conditions [1]. In ground robotics, F1TENTH provides standardized vehicles, circuits, metrics and baselines for continuous control and reinforcement learning [2], while full-scale competitions such as the Abu Dhabi Autonomous Racing League (A2RL) extend the same principle to head-to-head racing near the limits of vehicle handling [3]. In aerial robotics, gate-based competitions have driven the development of high-speed perception and control pipelines [4], and learning-based systems have more recently demonstrated autonomous drone-racing performance at champion level [5]. Across these domains, the value of racing is not limited to speed: a fixed task, explicit course geometry, common rules and measurable outcomes provide a practical basis for comparing, reproducing and improving complete autonomous systems. Underwater robotics provides a natural but substantially different setting for this type of evaluation. Satellite-based global positioning is unavailable underwater, inertial and Doppler-based navigation accumulates uncertainty over time, and visual navigation is affected by limited visibility, illumination variations and water conditions [6]. Acoustic sensing and communication can extend the observable range but introduce limited bandwidth, propagation delay and measurement uncertainty [7], while persistent currents and strongly damped vehicle dynamics further affect how a vehicle approaches, aligns with and crosses a gate. Underwater racing is therefore not simply a ground or aerial racing task transferred to a marine simulator.

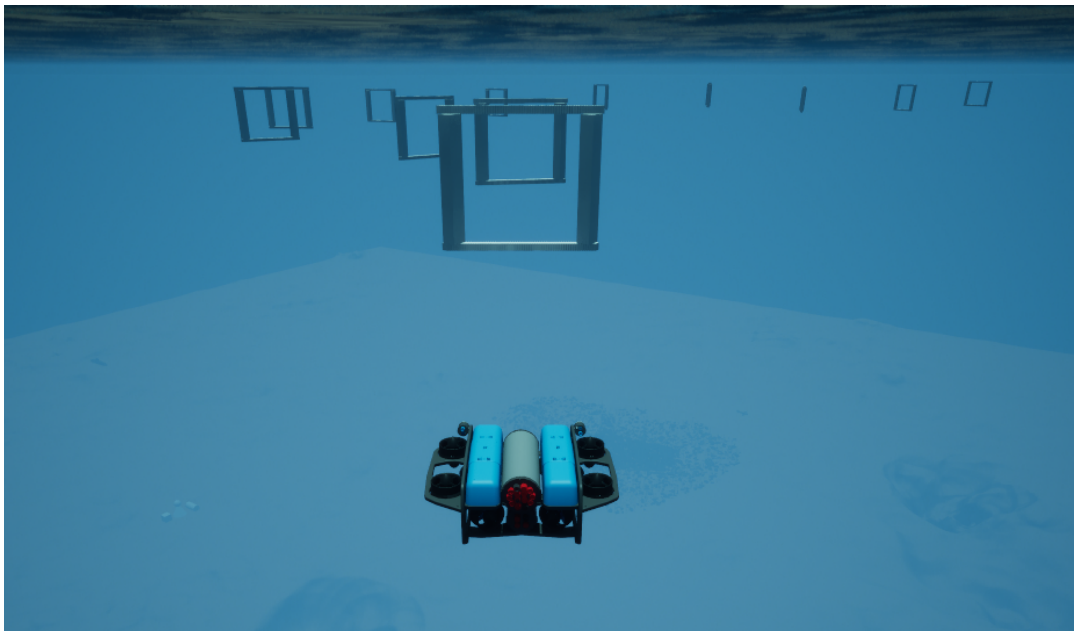

Figure 1: Underwater gate racing in Marine Race Arena. A BlueROV2-class vehicle approaches the ordered, beacon-marked gate line of an official circuit in HoloOcean. The controller sees this scene only through its forward camera, its depth, inertial and Doppler sensors and the acoustic packets it physically receives; the gate positions visible to the reader are never available to it.

A growing ecosystem of underwater simulators already provides the physical and sensor models required to develop autonomous systems. UUV Simulator introduced a Gazebo-based environment for underwater vehicle dynamics and multi-robot experimentation [8], while HoloOcean provides Unreal Engine environments, simulated underwater sensing and multi-agent support [9]. More recently, MarineGym has emphasized high-throughput simulation for learning-based underwater control [10]. These platforms provide increasingly capable simulation backends, but a simulator alone does not define a racing benchmark. It does not necessarily specify the ordered task to be completed, which information a controller is allowed to use, how gate crossings are validated, how penalties and timeouts are applied, or how different participants are evaluated under a common set of rules. This distinction is particularly important in simulation, where global vehicle pose, exact gate coordinates, the true current field or official course progress can simplify navigation substantially despite being unavailable to a deployed underwater vehicle. At the same time, relying only on the controller's internal estimate to determine task completion can hide perception, localization or sequencing failures. A meaningful benchmark should therefore separate the information available to autonomy from the information required for evaluation: controllers should operate from observations representative of onboard sensing, whereas privileged simulator state should remain confined to an independent evaluation mechanism.

This paper presents Marine Race Arena, a configurable HoloOcean benchmark for autonomous underwater gate racing and team-level fleet evaluation (Fig. 1). Marine Race Arena is implemented as a benchmark and race-management layer above the simulation backend. Complete race scenarios are defined through declarative configurations specifying the environment, ordered gates, vehicles, sensor profiles, acoustic beacons, environmental currents, obstacles, participants, starting conditions, timing rules and penalties. The reference implementation and the experiments reported in this work use HoloOcean with a BlueROV2-class vehicle [11], while simulator-specific sensing and actuation are isolated behind an adapter layer. A central feature of the framework is its controller-agnostic autonomy interface: the benchmark does not prescribe how a vehicle should navigate the course, but only defines the observations available to the participant and the commands it must provide. A controller can therefore be rule-based, optimization-based, learning-based or follow another autonomy paradigm without requiring changes to the race runner or evaluation logic. In parallel, a configurable automated referee uses privileged simulator state exclusively to validate ordered gate crossings, detect collisions and other rule violations, apply penalties and timeouts, determine race termination and compute the official results. Referee-side state and official course progress are never returned

to the controller. The same architecture extends to multi-vehicle experiments, where each vehicle retains independent autonomy and local state while the framework provides simultaneous or staggered starts, optional inter-vehicle communication, proximity monitoring, distributed leader–follower coordination and team-level scoring.

The release includes three ready-to-use gate-racing circuits with different geometric characteristics, together with configurable current profiles, obstacles and participant configurations. The common controller interface is demonstrated with two interpretable rule-based controllers and a recurrent reinforcement-learning controller, showing that substantially different autonomy approaches can be integrated without modifying the race-management or scoring logic. Structured event logs and machine-readable race summaries record both participant execution and official referee outcomes. The experimental evaluation exercises the framework across different circuits, environmental conditions and multi-vehicle configurations, with the purpose of assessing whether heterogeneous autonomy strategies and race scenarios can be executed and compared through the same observation interface, race definition and independent evaluation mechanism.

The main contributions of this work are as follows:

1. **A configurable benchmark for autonomous underwater gate racing.** Marine Race Arena defines complete race scenarios through declarative configurations covering tracks, ordered gates, vehicles, onboard sensors, acoustic beacons, environmental currents, obstacles, participants, starting conditions, timing, penalties and scoring. Three ready-to-use circuits are provided, while the same configuration model supports custom racing scenarios.
2. **A controller-agnostic autonomy interface.**
   Controllers interact with the benchmark through a common observation–action contract and can implement rule-based, optimization-based, learning-based or other autonomy strategies without modifying the race execution or evaluation logic. The release provides rule-based and reinforcement-learning controllers as reference implementations of this interface.
3. **A configurable race manager with an independent automated referee.** Race conditions and rules are defined independently from the controller, while privileged simulator state is restricted to the evaluation layer. The referee validates ordered gate crossings, detects violations, applies penalties and timeouts and computes official race results without exposing privileged progress information to autonomy.
4. **Single-vehicle and team-level multi-vehicle evaluation within the same framework.** Marine Race Arena supports independent per-vehicle autonomy, staggered or simultaneous starts, optional acoustic-inspired communication, proximity monitoring, distributed leader–follower coordination and team-level scoring while maintaining independent official progress for each participant.
5. **An experimental validation of the common benchmark interface.** The framework is exercised in HoloOcean across different race circuits, current conditions and multi-vehicle configurations, including both rule-based reference controllers and a reinforcement-learning controller integrated through the same observation and action interface. Structured outputs and automated validation procedures support repeatable evaluation and extension with new controllers and racing scenarios.

The remainder of the paper is organized as follows. Section 2 reviews underwater simulation, autonomous-racing benchmarks, underwater perception, multi-vehicle cooperation and learning-based marine control. Sections 3 and 4 introduce the benchmark model, runtime architecture, HoloOcean environment and observation interface. Section 5 describes onboard gate perception, followed by the reference controllers in Section 6 and the independent referee in Section 7. Section 8 presents multi-vehicle execution and coordination, while Section 9 describes the integration of learning-based control. Section 10 reports the experimental evaluation. The final sections discuss the results, limitations and reproducibility before concluding the paper.

2. Related Work

Underwater robotics has benefited from increasingly capable simulation environments, but these platforms differ substantially in scope and intended use. UUV Simulator extends Gazebo with underwater vehicle dynamics, thruster models, sensors and support for multi-robot scenarios [8]. Stonefish provides a marine-oriented simulator with geometry-based hydrodynamics, underwater rendering, sensors and ROS integration [12]. DAVE similarly builds

on Gazebo to support autonomous underwater missions with optical and acoustic sensing [13]. HoloOcean instead combines underwater vehicle simulation with Unreal Engine environments and a broad range of simulated onboard sensors [9]; its more recent development also supports multi-agent execution and extended sensing capabilities [14]. UNav-Sim focuses on visually realistic underwater navigation and synthetic-data generation in Unreal-based environments [15], while MarineGym targets high-throughput reinforcement-learning experiments using GPU-accelerated underwater simulation [10]. These systems provide the physical, visual and sensing substrates needed to develop underwater autonomy, but they do not by themselves define how a racing experiment should be configured, executed and evaluated.

This distinction has become increasingly relevant as robotics benchmarks move from isolated algorithm evaluation toward reproducible system-level testing. URoBench, for example, compares underwater simulators through common navigation and reinforcement-learning tasks [16]. More generally, robotics benchmarking literature has emphasized explicit task definitions, repeatable experimental conditions and measurable outcomes as necessary elements for meaningful comparison [17]. Marine Race Arena builds on the same principle, but shifts the focus from simulator comparison to race definition and execution. The simulator determines how vehicles and sensors evolve, while the benchmark determines the ordered task, participant interface, race conditions, penalties and official scoring.

Autonomous racing provides a useful reference for this type of separation. F1TENTH combines a standardized scaled vehicle with racing environments and interfaces suitable for both conventional control and reinforcement learning [2]. Its simulation ecosystem also supports multiple vehicles interacting in the same racing environment. At full scale, A2RL extends autonomous racing to head-to-head operation close to the dynamic limits of the vehicle [3]. A similar evolution occurred in aerial robotics, where AlphaPilot used ordered gates as a system-level task for high-speed perception, planning and control [4]. Subsequent work reinforced the use of gate racing as a complete autonomy benchmark [18], while Swift demonstrated that learned policies can reach very high levels of performance in autonomous drone racing [5]. In all these cases, the value of racing lies not only in speed, but in the presence of a defined course, explicit task progression and clear evaluation criteria.

CARLA provides another relevant example from autonomous driving. Its simulator can represent many simultaneously active vehicles [19], while the CARLA Autonomous Driving Leaderboard adds a participant interface and external evaluation procedure in which route completion and infractions are determined independently from the submitted agent. This separation between the information available to the autonomy system and the information used for evaluation is closely related to the design adopted in Marine Race Arena, although the task itself is different. CARLA targets road-driving scenarios rather than underwater racing and does not define a team-level fleet score. The comparison is therefore methodological rather than task-specific.

Gate traversal has also appeared in underwater robotics, although typically as part of a larger mission rather than as a reusable racing protocol. The FeelHippo platform demonstrates perception and control for underwater gate navigation [20]. Harada et al. investigate vision-based navigation for an underwater Gate Pass task [21]. Underwater gate racing adds further requirements because the vehicle must not only detect and cross a structure, but also identify the correct gate in an ordered sequence and continue the course without access to privileged race state. This is closely connected to the broader sensing limitations of underwater autonomy. Vision-based localization and control have been studied in real underwater environments where global positioning is unavailable [6], while perception-aware navigation considers how vehicle motion should preserve useful visual information [22]. Acoustic sensing provides complementary range and direction information, but underwater acoustic channels are affected by limited bandwidth, propagation delay, multipath and time-varying attenuation [7]. Marine Race Arena therefore keeps visual and acoustic interpretation inside the participant autonomy stack rather than using ground-truth gate geometry to resolve the target.

These communication constraints become even more important in multi-vehicle operation. Research on multi-AUV formation control has repeatedly shown that coordination quality depends strongly on communication availability and relative-state estimation [23]. Related reviews highlight the coupling between localization, communication and formation control in underwater fleets [24]. Marine Race Arena does not attempt to replace this literature with a new communication or formation-control method. Instead, it provides the race-level mechanisms needed to evaluate cooperative strategies: each vehicle maintains its own autonomy state, optional information is exchanged through an acoustic-inspired communication channel, and the race manager independently tracks progress, proximity events and team-level results. The included leader–follower strategy serves only as a reference controller for exercising this part of the framework.

Learning-based underwater control provides a further test of whether such a benchmark interface is sufficiently

Table 1: Positioning of Marine Race Arena against representative underwater simulators and racing benchmarks. Criteria are properties of the published system description, not quality judgements; the table deliberately omits hydrodynamic fidelity, sensor variety and training throughput, on which several listed systems exceed Marine Race Arena.

System	Domain	Physics and sensing	Gate race	Indep. referee	Obs. contract	Multi-robot	Team score
UUV Simulator [8]	Underwater	Gazebo, hydrodynamics	–	–	–	✓	–
Stonefish [12]	Underwater	Custom marine physics, sensors, ROS	–	–	–	–	–
DAVE [13]	Underwater	Gazebo, acoustic and optical sensing	–	–	–	✓	–
HoloOcean [14]	Underwater	Unreal Engine, sonar and onboard sensors	–	–	–	✓	–
URoBench [16]	Underwater	Cross-simulator navigation benchmark	–	–	✓	–	–
UNav-Sim [15]	Underwater	Unreal-based navigation simulation	–	–	–	–	–
MarineGym [10]	Underwater	Isaac Sim, GPU hydrodynamics, RL tasks	–	–	✓	*partial*	–
F1TENTH [2]	Ground	Scaled car, planar LiDAR, RL and control	✓	partial	✓	partial	no
AlphaPilot, Swift [4, 5]	Aerial	Quadrotor, vision-based gate racing	✓	partial	✓	no	no
CARLA [19]	Ground	Unreal Engine, urban sensing and tasks	partial	partial	✓	no	no
Marine Race Arena (this work)	Underwater	HoloOcean backend, BlueROV2, camera and acoustic sensing	✓	✓	✓	✓	✓

Gate race: an ordered gate course with timing is part of the defined task. *Indep. referee*: progress and violations are adjudicated by a component the participant cannot influence. *Obs. contract*: a documented observation interface; ✓ additionally denotes that ground-truth pose is excluded from control by the contract. *Team score*: a defined aggregate score over a group of vehicles. "Partial" marks a supported capability that is not a primary design contract of that system. "–" means that the cited public description does not specify the criterion; it is not a negative result. All listed underwater simulators supply physics and sensing that Marine Race Arena itself does not provide and depends on a backend for.

general. Platforms such as MarineGym expose observation and action spaces specifically designed for reinforcement-learning training [10], while common environment interfaces have long been used to decouple learning algorithms from the systems in which they are evaluated [25]. Marine Race Arena follows the same interface-oriented idea but has a different primary objective: it is not designed as a high-throughput training platform, but as a configurable racing and evaluation layer. A controller is required only to consume the observations allowed by the benchmark and return the expected vehicle commands. Its internal implementation can therefore be rule-based, optimization-based, learned or hybrid. The recurrent reinforcement-learning controller considered in this work is one example of this extensibility and is evaluated by the same race manager and referee used for the rule-based controllers.

Table 1 summarizes the resulting positioning. The comparison focuses only on benchmark-level properties relevant to this work: whether a system defines an explicit race task, exposes a participant-facing controller interface, restricts privileged simulator state, separates task evaluation from participant autonomy, supports multiple controllable vehicles in the same scenario and provides an explicit team-level result. These criteria are not measures of overall simulator quality. Several existing platforms offer greater physical fidelity, broader sensing or substantially higher training throughput than required here. Multi-vehicle simulation also does not automatically imply team-level evaluation: platforms such as UUV Simulator, DAVE, HoloOcean, F1TENTH and CARLA can represent several vehicles, but this is different from defining a common race protocol and an aggregate team result. Marine Race Arena combines these elements around a configurable underwater ordered-gate task, a controller-agnostic interface and an independent, configurable referee. Its contribution is therefore not a new underwater physics simulator, but a reusable racing and evaluation layer for comparing heterogeneous autonomy strategies under the same rules.

3. Benchmark Model and System Architecture

We specify Marine Race Arena as an *evaluation problem* rather than a controller-design problem: the benchmark fixes the world, the task and the scoring, while a participant supplies only a policy. This section defines the benchmark instance, the participant interface and the runtime architecture that enforce the separation between the two. Everything that follows — the perception front end of Section 5, the controllers of Section 6, the referee of Section 7, the fleet layer of Section 8 and the learned policies of Section 9 — is expressed in the terms introduced here.

3.1. Benchmark Instance

Definition 1 (Benchmark instance). A Marine Race Arena instance is the tuple

$$\mathcal{E} = (\mathcal{W}, \mathcal{G}, \mathcal{S}, \mathcal{F}, \mathcal{C}, \mathcal{O}, \mathcal{P}, \mathcal{R}), \tag{1}$$

where:

- $\mathcal{W}$ is the *world*: an axis-aligned bounds box $\mathcal{B} = [x_{\min}, x_{\max}]\times[y_{\min}, y_{\max}]\times[z_{\min}, z_{\max}] \subset \mathbb{R}^3$ (with z positive-up, so the navigable volume satisfies $z < 0$) together with the simulation environment selected through the adapter;
- $\mathcal{G} = (g_1, \ldots, g_M)$ is the *ordered* gate sequence; each gate is the tuple $g_k = (c_k, n_k, w_k, h_k)$ of centre $c_k \in \mathbb{R}^3$, passage-direction vector $n_k \in \mathbb{R}^3$ (nonzero, not necessarily unit length) and inner aperture width w_k and height h_k;
- $\mathcal{S}$ is the shared *start default*: a base spawn pose and an optional release policy that a participant inherits only when it does not specify its own;
- $\mathcal{F}$ is the finish condition: completion of the last gate of the last lap (Section 7);
- $\mathcal{C}$ is the current field $\mathbf{v}_c : \mathbb{R}^3 \times \mathbb{R}_{\geq 0} \to \mathbb{R}^3$ (Section 4);
- $\mathcal{O}$ is the obstacle set;
- $\mathcal{P} = \{1, \ldots, N\}$ is the participant set; participant i carries a vehicle, a sensor profile and a controller, together with an optional spawn pose and release delay $\tau_i \geq 0$ that override the start default of $\mathcal{S}$;
- $\mathcal{R}$ is the referee configuration: gate-validation parameters, penalty values and scoring rules.

Each gate's orientation derives solely from its passage direction n_g, giving a right-handed frame

$$\hat{n} = \frac{n_g}{\|n_g\|}, \quad \hat{r}_g = \frac{\hat{z} \times \hat{n}}{\|\hat{z} \times \hat{n}\|}, \quad \hat{s}_g = \hat{n} \times \hat{r}_g, \tag{2}$$

with world-up $\hat{z} = (0, 0, 1)$; if the passage direction is vertical ($\|\hat{z} \times \hat{n}\| \approx 0$), $\hat{r}_g$ falls back to the world $\hat{x}$ axis so the frame stays well defined. Deriving the frame from the passage direction alone, rather than from a separately authored gate rotation, removes a class of configuration error in which the visual orientation of a gate and the direction in which it must be traversed disagree. The basis is therefore unambiguous and independent of any visual rotation, and it is reused by the beacon placement (Section 4) and by the crossing test (Section 7).

3.2. Participant Interface

Definition 2 (Policy). A participant i implements a policy

$$\pi_i : o_{i,t} \mapsto u_{i,t}, \tag{3}$$

mapping the official observation $o_{i,t}$ (Section 4.6) to a normalized, high-level, body-frame command

$$u_{i,t} = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^\top \in [-1, 1]^4. \tag{4}$$

Table 2: Design requirements for the benchmark layer.

ID	Requirement
R1	Race configuration is declarative and reproducible from a single set of track JSON files.
R2	Official controller observations exclude ground-truth pose, simulator-only state and referee-derived course progress.
R3	Referee logic is independent of the participant controller and cannot be influenced by it.
R4	Gate dimensions, referee margins and track geometry are not changed to hide failures.
R5	Every run emits structured event logs and machine-readable summaries.
R6	Single-vehicle behaviour is unchanged when fleet mode is disabled.
R7	Fleet mode aggregates a team score while preserving per-vehicle diagnostics.

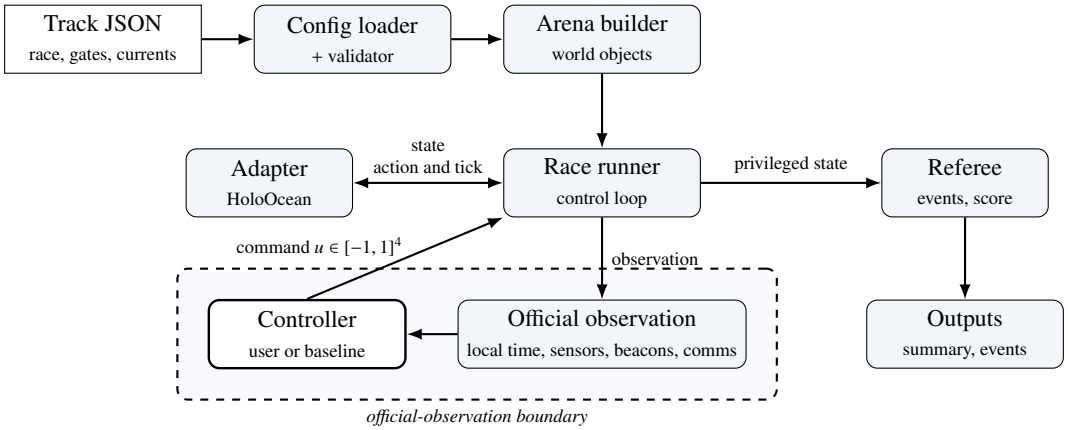

Figure 2: Runtime architecture and official-observation boundary between the controller and referee.

The action of (4) is the portable, constrained interface that an adapter maps to the selected vehicle. In fleet mode the policy may return $(u_{i,t}, m_{i,t})$, where $m_{i,t}$ is an optional controller-authored communication payload and only $u_{i,t}$ is a physical action. The reference HoloOcean adapter also accepts a direct eight-thruster `thrusters` action for its BlueROV2-class agent. Time is discretized at a fixed control timestep Δt, with $t_k = k\,\Delta t$. The adapter clamps $u_{i,t}$ to actuator limits and applies depth-bound safety before mapping it to the vehicle; in official mode the policy reads only the official observation $o_{i,t}$, whose contract excludes privileged state.

Fixing the action space at four normalized body-frame channels is what allows a rule-based controller, a hybrid controller and a neural policy to be compared without any of them gaining actuator authority the others lack. The learned policies of Section 9 emit exactly the vector of (4) and are subject to the same clamping and depth-safety rules as the reference controllers.

3.3. Design Requirements

The design follows the seven requirements in Table 2. They encode two commitments. First, scoring logic is independent of the participant and cannot be manipulated through privileged feedback. Second, enabling fleet mode must not silently change single-vehicle behaviour, so that a fleet result and a single-vehicle result remain comparable.

3.4. Runtime Architecture

Figure 3 complements the executable architecture with the information boundary used in the experiments: onboard observations and commands are separated from privileged referee state.

The runtime separates scenario construction, vehicle autonomy and race evaluation. A declarative track configuration defines the environment, the ordered gates, the beacon network, the disturbances, the participants and the scoring rules; after validation it becomes an arena instance connected to a simulator adapter. The adapter isolates sensing, actuation and time advancement, so the controller and referee interfaces are reusable across simulation backends. An engine-free adapter supports deterministic unit testing and is explicitly excluded from every reported result: each

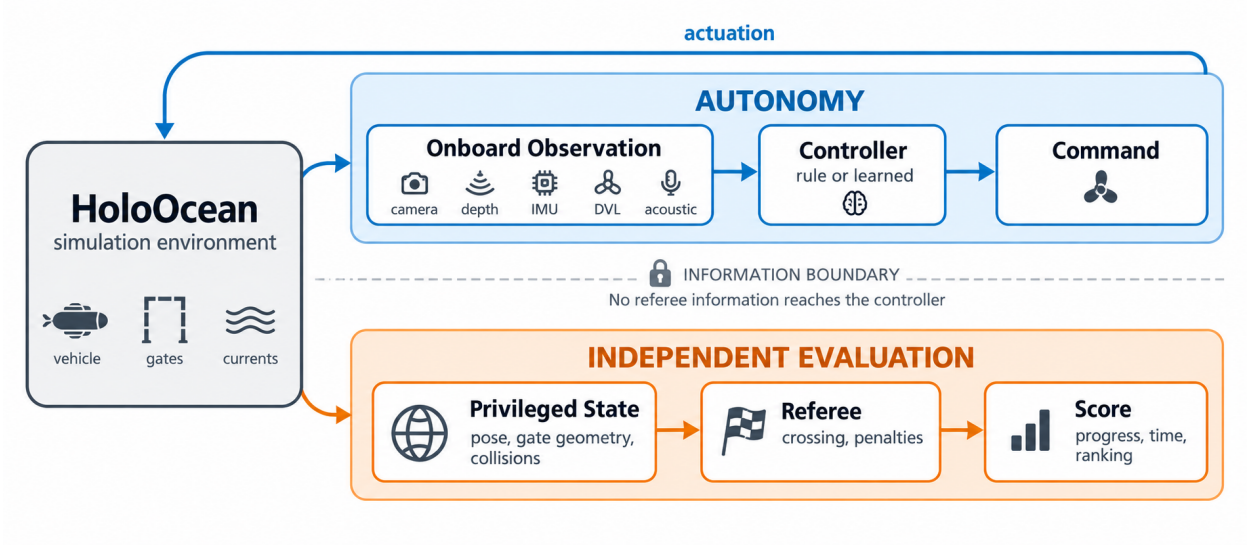

Figure 3: Conceptual information boundary: the participant receives allow-listed onboard observations and returns commands, while an independent referee uses privileged state only for adjudication and scoring.

experimental run records whether the fallback was permitted, and all results in this paper were produced with the fallback disabled and the native HoloOcean simulator backend confirmed in the manifest.

During execution the race runner advances the simulator and mediates vehicle–environment interaction. The controller receives an observation containing only participant-local time, allow-listed onboard measurements, simulated/modeled acoustic-channel packets and optional inter-vehicle messages. It returns a bounded motion command and cannot access simulator state, gate geometry, race progress or referee decisions, so navigation and course progression must be inferred locally. In parallel, the referee uses privileged simulator state to validate crossings and rule violations and to produce event logs, participant results and team-level scores. Referee-derived targets, progress and status are never returned to the controller; disagreements with the controller's local estimate are recorded for offline analysis rather than corrected, and Section 7.7 quantifies how large those disagreements actually are. Fig. 2 summarizes the separation and the controller-visible observation boundary.

4. Vehicle, Simulator, Sensing and the Observation Contract

This section describes the simulation backend, the vehicle and its sensors, the acoustic and current models, the declarative track configuration and the official observation contract that is enforced independently of the referee.

4.1. Simulator and Vehicle

The benchmark architecture is not tied to a particular vehicle: adapters provide simulator state, filtered sensing and action mapping to the common race and referee layers. The reference adapter used in every HoloOcean experiment reported here drives HoloOcean [9] with a BlueROV2-class agent [11] in direct eight-thruster mode. The simulator advances at 30 physics ticks per second.

Two control timesteps appear in this paper and are never pooled. The frozen benchmark matrix of Section 10.1 uses $\Delta t = 0.033$ s, so each control step corresponds to one physics tick and the effective control rate is ≈ 30 Hz. The current-free completion demonstration (Section 10.2) and the matched learned-controller benchmark (Section 10.6) use $\Delta t = 0.1$ s, i.e. 10 Hz control over the same 30 Hz physics. The slower rate reduces the number of policy evaluations per simulated second, which matters when a neural policy is in the loop, and it produces systematically longer completion times for the same controller on the same circuit. Every table in this paper states the control rate of the runs it reports, and no time from one rate is compared against a time from the other.

Table 3: Onboard sensor suite of the BlueROV2.

Sensor	Rate	Output and configuration
Depth	30 Hz	Scalar depth (m); noise $\sigma = 0$.
IMU	30 Hz	Linear acceleration and angular rate, with bias terms.
DVL	15 Hz	Body-frame velocity (m/s); beam elevation 22.5°, max range 50 m.
Front camera	30 Hz	RGB image, 640×480, 90° field of view.
Collision	optional	Boolean onboard contact flag when configured.

World-frame velocity, pose-derived heading, pose-derived depth and environmental-current telemetry are not controller-visible in official mode. Pose and velocity sensors may exist internally for simulation and referee evaluation only.

In the reference HoloOcean adapter, a high-level command $u = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^\top$ is mapped to the eight BlueROV2 thrusters by a fixed mixing matrix. The four vertical thrusters receive u^{heave}; the four horizontal thrusters receive

$$\tau_{1\text{-}4} = s_\tau \begin{bmatrix} u^{\text{surge}} + u^{\text{sway}} + \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} - \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} + u^{\text{sway}} - \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} + \kappa\, u^{\text{yaw}} \end{bmatrix}, \tag{5}$$

where $\kappa = 0.35$ is the yaw-mixing gain and s_τ the thruster scale; each component is clamped to the thruster limit. A controller may instead return the eight thruster values directly. The high-level interface is the default because it is simpler and vehicle-agnostic: a controller expresses intended body-frame motion while the framework handles mixing and depth safety. Direct thruster control remains available when finer actuator authority is required, but no result in this paper uses it.

4.2. Sensor Suite

Table 3 lists the onboard sensors. The official profile combines a forward RGB camera, a depth sensor, an IMU and a DVL; an onboard contact flag is also permitted when configured. Sensors providing world-frame velocity and ground-truth pose, location, rotation and dynamics exist internally for simulation and referee evaluation, but are rejected at the adapter boundary and cannot appear in the official controller observation (Section 4.6).

4.3. Acoustic Beacon Model

Let $(\hat{n}, \hat{r}_g, \hat{s}_g)$ be the right-handed gate frame of (2), with $\hat{n}$ the passage normal and $\hat{s}_g$ the gate-up axis. Each gate carries one beacon, placed at $b_g = c_g + \delta_n \hat{n} + \delta_r \hat{r}_g + \delta_s \hat{s}_g$ relative to the gate centre c_g (default offset $(0, 0, 0.35)$ m along $\hat{s}_g$). For a receiver at position p_r with heading ψ, let $\boldsymbol{\delta} = b_g - p_r$, let $\rho = \|\boldsymbol{\delta}\|$ be the geometric range and let $\tilde{\rho}$ be its noisy delivered measurement. The beacon reports $\tilde{\rho}$, a yaw-relative bearing, an elevation and a signal strength:

$$\rho = \|\boldsymbol{\delta}\|, \qquad \tilde{\rho} = \max(0,\, \rho + \eta_\rho), \quad \eta_\rho \sim \mathcal{N}(0, \sigma_\rho^2), \tag{6}$$

$$\beta = \text{wrap}(\,\text{atan2}(\delta_y, \delta_x) - \psi) + \eta_\beta, \tag{7}$$

$$\varepsilon = \text{atan2}(\delta_z,\, \sqrt{\delta_x^2 + \delta_y^2}) + \eta_\varepsilon, \tag{8}$$

$$\text{SS} = \max\Big(0,\, 1 - \tfrac{\tilde{\rho}}{\rho_{\max}}\Big),$$

where $\rho_{\max}$ is the beacon range and wrap$(\cdot)$ maps to $(-180°, 180°]$. Bearing and elevation carry independent angular perturbations $\eta_\beta, \eta_\varepsilon \sim \mathcal{N}(0, \sigma_\theta^2)$ with σ_θ in degrees, and the range carries a single perturbation η_ρ with σ_ρ in metres. Each is drawn once per packet, and the signal strength derives from the same noisy range $\tilde{\rho}$, so no cleaner distance leaks back as a side channel. A transmission is not received beyond $\rho_{\max}$ or when a Bernoulli dropout fires. The official tracks set σ_θ and σ_ρ to a common per-track value in $\{0.2, 0.45, 0.6\}$ (degrees and metres respectively) and dropout probabilities up to 0.04.

Every ordered gate has one independent periodic transmitter with a unique contiguous identifier $\text{B}01, \ldots, \text{B}M$. All beacons transmit regardless of referee state; at each update a vehicle receives every in-range, non-dropped packet.

The framework does not tell the controller which packet corresponds to its next gate, and does not suppress the others: deciding which received identifier to pursue is part of the participant's autonomy (Sections 5 and 6). Packet randomness is seeded by the run seed, the transmitter, the receiver and the transmission index, which makes reception independent of participant count and call order and therefore keeps a fleet run reproducible.

4.4. Current Field Model

The current field is a sum of analytic primitives evaluated at the vehicle position p and time t,

$$\mathbf{v}_c(p,t) = \sum_j \mathbf{v}_j(p,t), \tag{9}$$

where each primitive is one of the following. A *constant* term contributes a fixed velocity $\mathbf{v}_0$. A *localized jet* centred at c_j with radius R_j contributes, for $\|p - c_j\| \le R_j$,

$$\mathbf{v}_j = \mathbf{v}_0\,\omega(d_j), \qquad \omega(s) = \begin{cases} \max(0, 1 - \frac{s}{R_j}) & \text{linear,} \\ \exp(-\frac{s^2}{2\sigma_j^2}) & \text{Gaussian,} \end{cases} \tag{10}$$

where $d_j = \|p - c_j\|$ is the jet distance and $\sigma_j = \max(R_j/2, 10^{-6})$ the jet width; the contribution is zero outside R_j. A *sinusoidal* term modulates one axis, $v_{j,\text{axis}}(t) = v_{0,\text{axis}} + A\sin(2\pi f t + \phi)$. A *vortex* of radius R_j, tangential speed V_t and vertical speed V_z contributes, with $r = \sqrt{\Delta x^2 + \Delta y^2}$ and tangent $\hat{\theta} = (-\Delta y, \Delta x)/r$,

$$\mathbf{v}_j = (\hat{\theta}_x\, V_t\, \omega(r),\ \hat{\theta}_y\, V_t\, \omega(r),\ V_z\, \omega(r)). \tag{11}$$

The provided `strong` profile superposes a constant set-flow of $[0.75, 1.05, 0]$ m/s, two Gaussian jets, a vortex and a vertical sinusoid; the `medium` profile scales these magnitudes by 0.5. Both named profiles are defined relative to the gate layout and are available on each official circuit. The HoloOcean adapter evaluates the field at every vehicle before each control step. Crucially, the current field is applied physically and is *not* reported to the controller: a participant must infer the disturbance from its own onboard measurements, in practice from the difference between commanded motion and body-frame DVL velocity.

4.5. Declarative Track Configuration

A track is a self-contained JSON file. Its sections define the race timing, benchmark task, world, ordered gates, sensors, beacons, currents, obstacles, participants and referee. The selected task supplies the corresponding validation contract; Listing 1 shows a representative excerpt.

Listing 1: Representative excerpt of a track configuration.

```
"track": {
  "gate_inner_size_m": [1.5, 1.5],
  "gate_sequence": ["G01", "G02", "..."]
},
"gates": [
  { "id": "G01", "position": [4.0, -2.0, -3.5],
    "rotation_rpy_deg": [0.0, 0.0, 0.0],
    "passage_direction": [1.0, 0.0, 0.0] }
],
"beacon": {
  "range_m": 90.0,
  "angular_noise_std_deg": 0.2,
  "range_noise_std_m": 0.2,
  "dropout_probability": 0.0,
  "update_rate_hz": 10.0
},
"participants": [
  { "id": "bluerov2_01", "vehicle": "BlueROV2",
```

Table 4: Official track configurations.

Track file	Track name	Gates	Laps	Total gates	Length (m)	Intended use
`marine_race_horseshoe_bay.json`	Marine Race Horseshoe Bay	12	1	12	93.8	Clean-gate horseshoe baseline
`marine_race_vertical_serpent.json`	Marine Race Vertical Serpent	17	1	17	118.3	Vertical and slalom gate sequencing
`marine_race_mixed_endurance.json`	Marine Race Mixed Endurance	22	1	22	206.3	Longer endurance and current-oriented layout

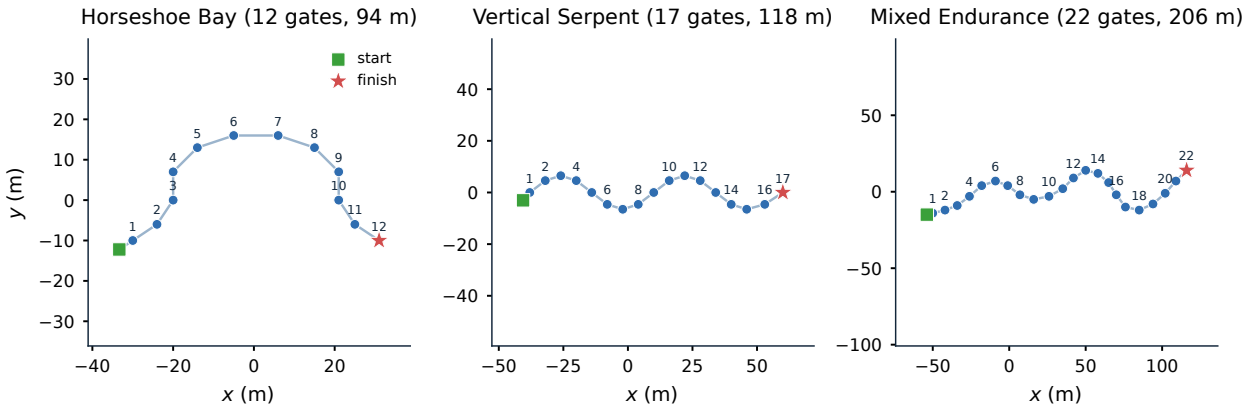

Figure 4: Top-down (x, y) layouts of the three official circuits, drawn from the released track JSON: ordered gates, start (square) and finish (star). The circuits vary in length (93.8, 118.3 and 206.3 m), in gate count (12, 17 and 22) and in vertical structure.

```
    "controller": "rule_gate_baseline",
    "start_delay_s": 0.0 }
],
"referee": {
  "penalties": { "minor_collision_s": 5.0 },
  "gate_validation": {
    "vehicle_clearance_margin_m": 0.1 }
}
```

The three official circuits are summarized in Table 4 and their layouts shown in Fig. 4. They are deliberately heterogeneous: Horseshoe Bay is a short, largely planar baseline; Vertical Serpent forces repeated vertical transitions and slalom sequencing; Mixed Endurance is more than twice as long as Horseshoe Bay and combines both. All three use 1.5 m × 1.5 m apertures, and neither the aperture size nor the referee margin was changed at any point of this study (requirement R4).

The `obstacle_gate` task accepts explicit fixed obstacles or deterministic random boxes generated between consecutive gates with a configured density and minimum clearance. The HoloOcean adapter records the requested and physically spawned counts, and an obstacle construction check is accepted only when they match. This verifies that obstacles can be instantiated on a circuit; it is deliberately separate from any claim that a reference controller avoids them, and no obstacle-avoidance result is reported in this paper.

4.6. Official Observation Contract

The controller `step` function receives a dictionary with the fields in Table 5. The builder is structurally independent of the referee: its signature contains the track configuration, the arena, the adapter and the participant state, but no referee object, so referee-derived quantities are not merely omitted by convention but are unreachable from the code path that constructs the observation. The builder computes participant-local elapsed time, requests only allow-listed onboard sensors from the adapter and queries the beacon field for packets delivered by simulated periodic

Table 5: Top-level fields of the official observation.

Field	Meaning
`local_time_s`	Elapsed time since this vehicle's release; it is not official race time.
`sensors`	Filtered onboard dictionary containing only the configured camera, depth, IMU, DVL and optional collision measurements.
`beacons`	List of packets delivered by the simulated acoustic channel from independent transmitters; it may be empty or contain several identifiers, and no field marks which one is the next gate.
`comms`	Present only when the optional inter-vehicle acoustic channel is enabled: an `inbox` of messages received from teammates, each with the sender identifier, the controller-authored payload and a receiver-local reception timestamp (Section 8).

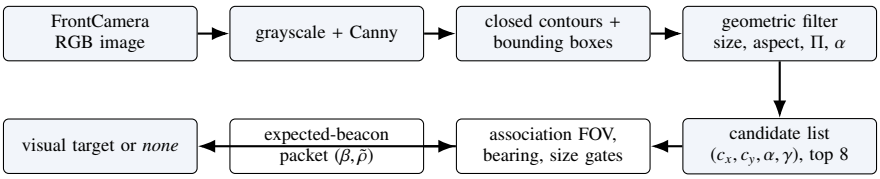

Figure 5: The onboard gate-perception front end. Camera pixels produce an unlabelled candidate list; the controller's own expected-beacon packet supplies the bearing that selects among them. No simulator gate pose, global vehicle pose or referee target enters any stage.

transmissions. Numeric arrays are made JSON-safe while image arrays are preserved unchanged. When the optional inter-vehicle acoustic channel is enabled (Section 8), the observation additionally carries a `comms` inbox whose payloads are authored by another controller from its own legal information.

Three quantities are worth naming explicitly because their absence defines the task. The controller never receives the global vehicle pose, so it cannot localize itself in the world frame. It never receives gate positions, gate normals or aperture dimensions from the simulator, so the geometry of the course must be inferred from beacons and images. And it never receives the referee's notion of which gate is next, whether the previous crossing was valid, or how many gates remain, so course progression is a controller-side estimate that can be, and sometimes is, wrong.

5. Onboard Gate Perception and Target Association

A racing benchmark is only as strict as its weakest information leak. The observation contract of Section 4.6 withholds gate geometry and referee progress, but it delivers a camera image that may contain several gates and a list of acoustic packets whose identifiers carry no ordering information. Deciding *which* visible rectangle is the gate the vehicle must pass next is therefore not a preprocessing step that the framework can perform on the participant's behalf: doing so would silently reintroduce course knowledge into the control loop. This section describes the onboard front end that the reference controllers use to make that decision, states the assumptions it depends on, and reports what it achieves and where it fails.

Figure 6 gives a compact conceptual view of the camera–acoustic association path and its selected gate target.

5.1. Gate Detection from the Forward Camera

Motivation. The gates are rendered as square frames of light-coloured bars. A naive bright-pixel segmentation is unusable in this environment: HoloOcean tints the public `white` prop material strongly with the water column, so a gate bar can be *darker* than the surrounding water and the seabed, and a generic brightness or saturation mask selects most of the scene. The detector therefore keys on the one property that survives the tint, namely that a gate projects to a near-rectangular closed contour of bounded aspect ratio, and validates candidates geometrically rather than photometrically.

Forward process. Given a FrontCamera image of width W and height H, the detector proceeds as follows.

1. **Preprocessing.** The RGB frame is normalized to 8-bit, converted to grayscale and filtered with a Canny edge detector with hysteresis thresholds $(30, 80)$.

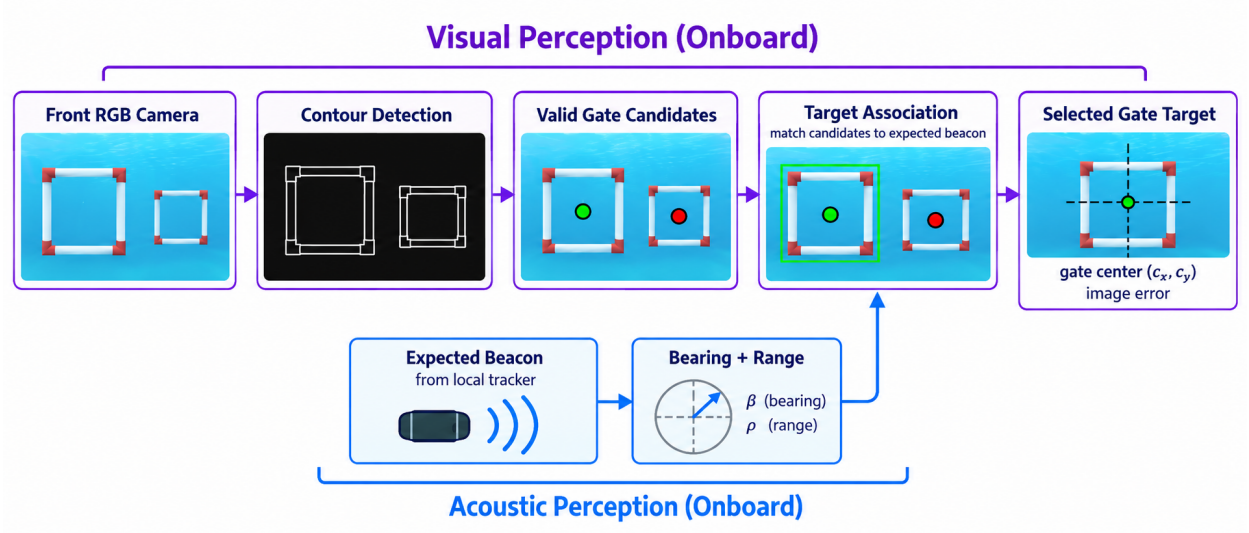

Figure 6: Conceptual onboard perception path from camera and acoustic-channel observations to candidate gates, beacon association and the selected target.

2. **Contour extraction.** Closed contours are extracted from the edge map, and each contour is reduced to its axis-aligned bounding box $(x, y, w_{\mathrm{px}}, h_{\mathrm{px}})$.
3. **Geometric filtering.** A candidate survives only if it is large enough to be a gate and shaped like one: $w_{\mathrm{px}} \geq \max(12, 0.05W)$ and $h_{\mathrm{px}} \geq \max(12, 0.05H)$; the box occupies less than 0.92 of either image dimension, which rejects frame-filling background structure; the aspect ratio $a = w_{\mathrm{px}}/h_{\mathrm{px}}$ lies in $[0.45, 2.40]$; the bounded perimeter ratio $\Pi = L_{\mathrm{contour}}/(2(w_{\mathrm{px}} + h_{\mathrm{px}}))$ lies in $[0.50, 2.20]$, which rejects filled background regions and irregular surface reflections whose contour length is inconsistent with their bounding box; and the area fraction $\alpha = w_{\mathrm{px}}h_{\mathrm{px}}/(WH)$ is at least 0.008.
4. **Normalized image geometry.** A surviving candidate is reported in image-normalized coordinates centred on the optical axis,
$$c_x = \frac{(x + \frac{1}{2}w_{\mathrm{px}}) - \frac{1}{2}W}{\frac{1}{2}W}, \qquad c_y = \frac{(y + \frac{1}{2}h_{\mathrm{px}}) - \frac{1}{2}H}{\frac{1}{2}H}, \tag{12}$$
both clamped to $[-1, 1]$, together with the area fraction α and the width and height fractions w_{px}/W and h_{px}/H. The pair (c_x, c_y) is the normalized image error with respect to the camera centre, and is the quantity the visual servo drives to zero.
5. **Confidence.** Each candidate carries a bounded confidence combining four normalized scores — rectangularity, aspect plausibility, apparent size and centredness,
$$\gamma = 0.38\, s_\Pi + 0.24\, s_a + 0.20\, s_\alpha + 0.18\, s_c \in [0, 1], \tag{13}$$
with $s_\Pi = 1 - |\Pi - 1|/1.20$, $s_a = 1 - |\log a|/\log 3$, $s_\alpha = \alpha/0.10$ and $s_c = 1 - 0.45|c_x| - 0.30|c_y|$, each clamped to $[0, 1]$. Candidates with $\gamma < 0.38$ are discarded.
6. **Multiple candidates.** Surviving candidates are sorted by confidence, deduplicated when their normalized centres differ by less than 0.04 in both axes, and the strongest eight are returned. A parallel coarse-grid colour-blob path runs alongside the contour path and contributes additional candidates, which keeps the detector usable in minimal installations where OpenCV is unavailable.

Technical advantage. Reporting a *list* of candidates with per-candidate geometry, rather than a single best detection, is what makes the next stage possible. A detector that silently returns its most confident rectangle would resolve the association question implicitly, and would resolve it wrongly whenever the most confident rectangle belongs to a later gate — which is common on the official circuits, where consecutive gates are frequently visible together.

5.2. Acoustic–Visual Target Association

Motivation. The controller knows which beacon identifier it is currently pursuing, because it maintains that state itself (Section 6), and it receives a noisy bearing β and range $\tilde{\rho}$ for that identifier from (6). It does not know which image rectangle that beacon belongs to. Association is the step that binds the two modalities, and it is also the step where an incorrect commitment is most expensive: locking onto the wrong gate produces a confident, well-servoed approach to a gate the referee will score as a miss.

Forward process. Let $\mathcal{T}$ be the candidate list, and let β and $\tilde{\rho}$ be the bearing and range of the expected beacon, or $\varnothing$ when no packet for that identifier was received.

1. **Field-of-view gate.** If $|\beta| > 70^\circ$ the association returns nothing. With a 90° horizontal field of view, a target that far off the nose cannot be the centre of the expected gate. This case matters immediately after a passage, when the just-cleared beacon lies behind the vehicle while the next gate is already visible ahead; accepting the forward rectangle there would change target identity without the tracker ever deciding to advance.
2. **Implied bearing.** Each candidate is assigned the body bearing implied by its horizontal image position,
$$\hat{\beta}(c_x) = -\arctan(c_x), \tag{14}$$
in degrees, with positive bearing to camera-left, and a mismatch $m = |\hat{\beta}(c_x) - \beta|$.
3. **Near-field size gate.** When $\tilde{\rho} < 1.6\,\mathrm{m}$, a $1.5\,\mathrm{m}$ aperture must subtend a substantial part of the image, so candidates are kept only if $\alpha \geq 0.05$ or either linear fraction is at least 0.30. A small, complete rectangle at that acoustic range is a later gate, not a reappearance of the current one.
4. **Conflict rejection.** If any candidate satisfies $m \leq 32^\circ$, every candidate with $m > 32^\circ$ is rejected outright. The acoustic bearing is thus a hard identity gate whenever it is informative, not merely a soft score term, which prevents a large blob spanning several visible gates from outscoring the correct one.
5. **Scoring.** Surviving candidates are ranked by confidence plus a centredness term and a size term, minus a bearing penalty $0.050\,m$. The size term is deliberately dominated by a *relative* component, $s_{\mathrm{rel}} = \sqrt{\alpha/\alpha_{\max}}$ where $\alpha_{\max}$ is the largest area fraction in the list, because an absolute size score saturates when two gates are both reasonably large and then lets the more centred, farther gate win. When $|\beta| > 25^\circ$ an additional side test is applied: the candidate must lie on the image side the acoustic bearing indicates, by at least 0.10 normalized units (0.18 beyond 45°).
6. **Fallback.** With no expected-beacon bearing available, the association degrades to a bearing-free default that prefers large, confident, centred candidates. With no surviving candidate at all, the front end reports that no visual target is available for this frame, and the controller must proceed on acoustic and inertial evidence alone.

Technical advantage. Because association is driven by the controller's own expected identifier and by a bearing delivered through the simulated acoustic channel, the front end never needs, and never receives, a ground-truth answer. Nothing in the pipeline consults simulator gate poses, the global vehicle pose, or the referee's target index. The consequence is that a wrong association is a genuine autonomy failure that the referee will score, which is exactly the property a benchmark needs.

5.3. An Exploratory Pose-Aware Extension

The centre-only representation is geometrically ambiguous, so we also implemented an exploratory pose-aware front end based on the detected quadrilateral and the known aperture. It is reported as an extension of the perception stack, not as a component of the main results.

The extension estimates plane orientation and distance when the geometry is sufficiently well conditioned. Implementation details and masks are retained in the repository, but the resulting estimates are not supplied to the controllers evaluated here.

Measured limits. On a dedicated native-HoloOcean simulator setup with $\pm 25^\circ / \pm 45^\circ$ oblique views, pose was available in only 11.7% of frames and the median plane-yaw error was 48.2°; orientation and yaw-sign accuracy were both 0.00. These metrics are insufficient for the main controller claims.

All main results therefore use the centre-only detection and acoustic–visual association front end. The pose extension is retained as an implemented diagnostic and an explicit limitation (Section 12).

Table 6: Onboard perception audit along the reference controller's own trajectory on the three official circuits, native HoloOcean simulator backend, 60 frames per circuit. Ground truth was used only offline to score the front end and never entered the control loop.

	Availability			Plane orientation error			Association	
Circuit	Det.	4-corner	PnP	med. (°)	p90 (°)	$>30°$	multi	correct
Horseshoe Bay	0.900	0.817	0.517	0.018	1.204	0.000	7	1.000
Vertical Serpent	0.883	0.783	0.483	1.129	4.287	0.000	4	1.000
Mixed Endurance	0.917	0.833	0.533	0.064	0.064	0.000	11	1.000

Det.: fraction of frames with at least one accepted detection. *4-corner*: fraction with a validated aperture quadrilateral. *PnP*: fraction with a metric planar-PnP solution. *multi*: number of frames in which more than one candidate was present. *correct*: fraction of multi-candidate frames whose online association agreed with an offline referee selection within a normalized centre tolerance of 0.25; the online target and the offline selection agreed on every frame of all three circuits. No circuit recorded a wrong orientation sign, a gross orientation error above 30°, a frame-to-frame yaw jump above 20°, or an unintended online target switch. On Mixed Endurance the expected gate was outside the field of view in 3 of the 60 frames; the in-view detection and four-corner rates there are 0.912 and 0.825. This is a characterization of the perception component from a diagnostic capture, not a controller performance claim.

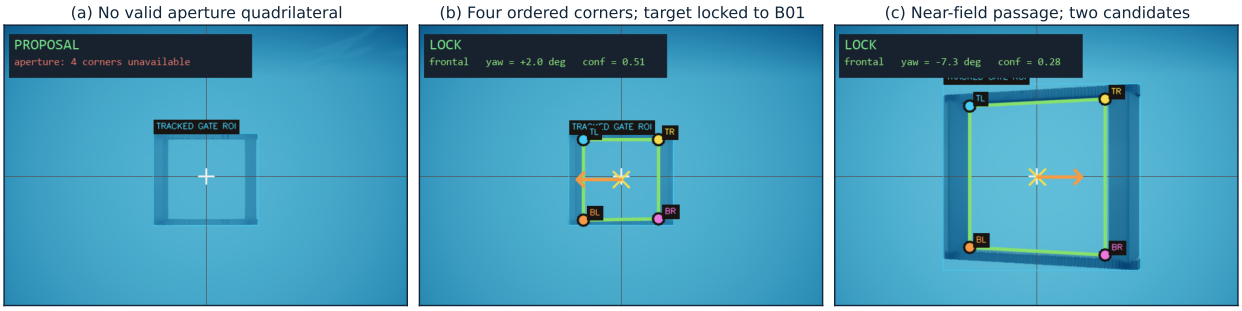

Figure 7: Onboard gate perception in the native HoloOcean simulator backend, one representative state per official circuit. Each panel shows the forward-camera frame with the detector's own overlays: the tracked aperture region, the four canonically ordered corners (TL, TR, BR, BL), the image-centre crosshair, the estimated gate centre and the image error vector of (12) that the visual servo drives to zero. (a) Mixed Endurance: a target is detected but no valid aperture quadrilateral is available, so the front end degrades to the centre-only estimate and reports an unknown orientation. (b) Horseshoe Bay: four validated corners with the target associated to the expected beacon `B01` at 4.39 m and −0.0° bearing, tracker phase VISUAL_ALIGN. (c) Vertical Serpent: a near-field passage at 2.25 m with two raw candidates, in which the association of Section 5.2 retains the correct target, tracker phase VERIFY_EXIT. Nothing shown here is derived from simulator gate poses or from the referee. Panel chrome from the interactive viewer has been cropped and its state badge redrawn in English; the detector overlays and all quoted values are unmodified.

5.4. Perception Audit on the Official Circuits

The oblique rig above is an adversarial probe, not the operating regime. To characterize the front end where the controller actually uses it, we instrumented the perception stack along the reference controller's own trajectory on all three official circuits in the native HoloOcean simulator backend, recording 60 frames per circuit (seeds 67000–67002). Ground truth was used only offline to score the detector and never entered the control loop. Table 6 reports the result, and Fig. 7 shows three representative states.

Three observations follow. First, detection is available in 88.3–91.7% of frames and a valid four-corner aperture in 78.3–83.3%, so the front end is usable but intermittent; the controller must tolerate gaps rather than assume a detection every frame. Second, metric PnP succeeds in only about half of frames (48.3–53.3%), which is consistent with the negative pose result above and is why the projective fallback matters. Third, and most relevant to the association design, the target chosen online agreed with an offline referee selection in every frame on all three circuits, the multi-candidate association was correct in every multi-candidate frame (7, 4 and 11 frames respectively), and no spurious online target switch occurred. Under the near-frontal viewing geometry that the centre-then-commit approach maintains, the plane-orientation estimate is also accurate (median error 0.018–1.129°, no error above 30°, no sign error, no frame-to-frame yaw jump above 20°).

The contrast between this table and the oblique rig is the substantive finding. The pose front end is not uniformly weak; it is accurate in the near-frontal regime and fails under strong obliquity. Because the reference controller

actively maintains a near-frontal approach, it operates inside the regime where the front end works, which explains both why the rule-based baselines complete circuits and why the pose features are not needed to do so. This audit is a characterization of the perception component and, being based on 60 frames per circuit from a diagnostic capture, is reported as such; no completion claim in Section 10 rests on it.

6. Controller Interface and Rule-Based Reference Controllers

A central usability goal of Marine Race Arena is that an autonomy stack can be integrated through a small interface without modifying the runner, the referee or the adapter. This section specifies that interface, the dynamic loading mechanism, the controller-local course tracker and the two rule-based reference controllers whose comparison anchors the evaluation.

6.1. Controller Interface

A controller implements three methods: `reset(mission_info)`, called once before the run with a minimal static assignment; `step(observation)`, which receives the official observation of (3) at every control tick and returns the command of (4); and `close()`, called at teardown. The contract is duck-typed, so any object exposing the three callables is accepted and a participant need not depend on framework base classes. A manual controller may raise a dedicated stop exception from `step` to end a run cleanly; any other exception is recorded as a controller error and terminates the participant. Listing 2 shows a minimal example.

Listing 2: Minimal custom controller.

```
class MyController:
    def reset(self, mission_info):
        self.tracker = LocalCourseTracker(
            initial_beacon_id=mission_info["initial_beacon_id"],
            total_beacons=mission_info["total_beacons"],
            laps=mission_info["laps"])

    def step(self, observation):
        sensors = observation["sensors"]
        state = self.tracker.update(
            local_time_s=observation["local_time_s"],
            beacons=observation["beacons"],
            camera_image=sensors.get("FrontCamera"),
            dvl_velocity=sensors.get("DVLSensor"))
        if state.finished:
            return {"surge": 0.0, "sway": 0.0, "heave": 0.0, "yaw": 0.0}
        # ... map state.beacon and camera to a command ...
        return {"surge": 0.3, "sway": 0.0, "heave": 0.0, "yaw": 0.0}

    def close(self):
        pass
```

The `mission_info` passed to `reset` is deliberately minimal: the initial beacon identifier, the number of beacons and the lap count. It contains no gate positions, no course length in metres and no ordering beyond the starting identifier. The high-level command fields are listed in Table 7; values are normalized to $[-1, 1]$ and clamped by the framework before the action mapping of (5). In fleet mode a controller may additionally return a small `message` payload alongside its command and read incoming teammate messages from the observation.

6.2. Dynamic Loading

A controller is selected at run time by a built-in alias, a dotted module path with an explicit class, or a file path and class name imported dynamically. The runner also provides manual controllers for inspection and data collection. An external policy is therefore evaluated without modifying the package:

Table 7: High-level controller command fields of (4).

Field	Interpretation
`surge`	Longitudinal body-frame thrust; positive moves forward.
`sway`	Lateral body-frame thrust; sign follows the adapter mixing of (5).
`heave`	Vertical thrust; additionally constrained by depth-safety logic near the world bounds.
`yaw`	Yaw-rate command; sign follows the adapter mixing of (5).

```
python -m marine_race_arena.scripts.run_marine_race \
  --track .../marine_race_horseshoe_bay.json \
  --benchmark-task clean_gate \
  --controller path/to/my_controller.py \
  --controller-class MyController \
  --adapter holoocean --official --headless
```

The same mechanism loads the learned policies of Section 9: a thin wrapper exposes the three interface methods, encodes the official observation into the policy's input vector and returns the policy's output as the command of (4). No part of the runner, referee or adapter is aware that the controller is neural.

6.3. Controller-Local Course Progression

Motivation. Because the observation carries no course progress (Section 4.6), a controller that must pass M gates in order has to maintain that ordering itself. The framework supplies a reusable component, the `LocalCourseTracker`, that both reference controllers own; the runner supplies no progression signal of any kind.

Module design. The tracker initializes from the public mission assignment `initial_beacon_id` and maintains a local expected beacon identifier, a completed count, a lap index and a status through the sequence

$$\text{SEARCH} \rightarrow \text{APPROACH} \rightarrow \text{VISUAL\_ALIGN} \rightarrow \text{COMMIT} \rightarrow \text{VERIFY\_EXIT} \rightarrow \text{ADVANCE},$$

ending in FINISHED after the final locally confirmed beacon. At each tick it filters the received packet list for its own expected identifier — packets from other transmitters are neither labelled nor selected by the framework — and combines that packet with the associated visual target of Section 5.2 and with body-frame DVL displacement.

Advancement conditions. The tracker enters and completes a passage only when persistent camera, acoustic and DVL evidence agree with the expected beacon. Five conditions individually block an advancement: a temporary visual loss, a missing acoustic packet, an isolated range jump, a range rise without a prior close approach, and a commit without forward DVL displacement. Incomplete or inconsistent evidence returns the tracker to local acquisition without advancing. After each non-final advancement it holds an exit-clearance phase; after the final advancement it enters FINISHED directly and commands zero motion. Fig. 8 shows the state machine.

Technical advantage and its cost. Requiring agreement across three modalities is what keeps the local estimate close to the referee's: over the frozen benchmark matrix the tracker produced 1472 advancements against the referee's 1474, with 11 false and 7 missed (Section 7.7). The cost is latency — the tracker confirms a passage a median of 0.858 s after the referee records it — and this lag is visible in the controller's behaviour rather than hidden, because the exit-clearance phase begins only once the tracker is convinced. Controller-local snapshots and independent referee crossings are logged separately for offline comparison and are never fed back into control.

The two reference controllers and their local-progression logic were designed and validated for the gate geometry, beacon placement, sensing configuration and vehicle dynamics of the three official circuits used in this study; they are interpretable baselines, not tuned optimal policies, and we make no claim that their thresholds transfer unchanged to a different course family.

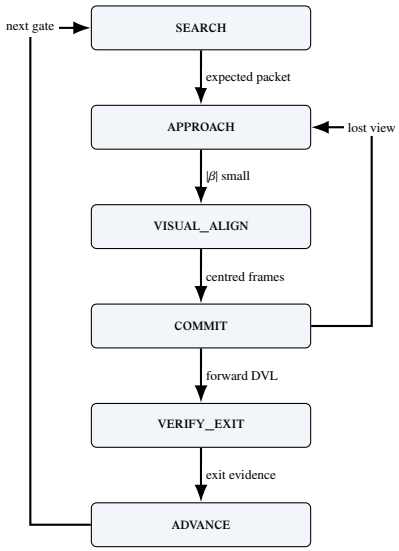

Figure 8: The LocalCourseTracker state machine shared by both reference controllers. ADVANCE loops back to SEARCH for the next gate and reaches FINISHED after the last locally confirmed passage. No transition consumes referee state.

6.4. Continuous Servo

Both reference controllers are deterministic and interpretable rule-based baselines rather than optimized policies, and both use exactly the same onboard inputs: received acoustic packets, the forward RGB camera, depth and IMU measurements and body-frame DVL velocity. They combine acoustic guidance, visual centring, depth hold and DVL-based passage verification through the shared tracker of Fig. 8.

Continuous Servo steers toward the expected beacon using its bearing and elevation, and refines that alignment with the associated visual target as the gate becomes visible. Depth feedback maintains the vertical reference and DVL motion provides the evidence that the vehicle has crossed and cleared the gate. Its defining behaviour appears during COMMIT and VERIFY_EXIT: the controller continues applying bounded visual corrections while moving through the aperture, and therefore keeps servoing toward the observed image centre throughout the passage.

6.5. Center-then-Commit

Motivation. Continuous visual correction is well behaved at range and poorly behaved at contact distance. As the vehicle enters the aperture the gate frame leaves the field of view piecewise: bars clip at the image border, the detected contour becomes a partial rectangle, and the centroid of that partial rectangle moves rapidly even though the vehicle is on line. A controller that keeps chasing the centroid then commands large late corrections precisely when the aperture margin is smallest.

Module design. Center-then-Commit retains the same observations, the same acoustic and visual guidance, the same shared tracker and the same local confirmation criterion as Continuous Servo. The only difference is the gate-passage strategy: after establishing a stable visual–acoustic lock, it stops chasing subsequent image-centroid changes during COMMIT and VERIFY_EXIT, and instead advances through the aperture while holding the locked depth and attitude from onboard depth and IMU feedback. The same DVL-based evidence verifies the passage.

Technical advantage. Because the observations, the tracker and the confirmation logic are unchanged, the matched comparison of Section 10.3 isolates one variable: continuous visual correction versus a stabilized committed passage. This is the cleanest ablation the benchmark supports, and Section 10.3 shows that its outcome is condition-dependent rather than uniform — the staged passage costs a little speed on a short clean circuit and buys substantial reliability on a long one and under disturbance.

The same distinction is summarized in Fig. 10, which also contrasts continuous servoing with center-then-commit passage behaviour.

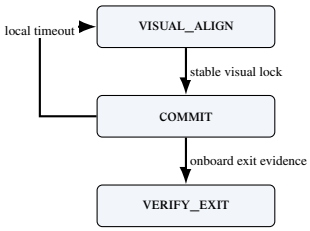

Figure 9: Center-then-commit passage in the COMMIT and VERIFY_EXIT states, with controller-local confirmation and recovery.

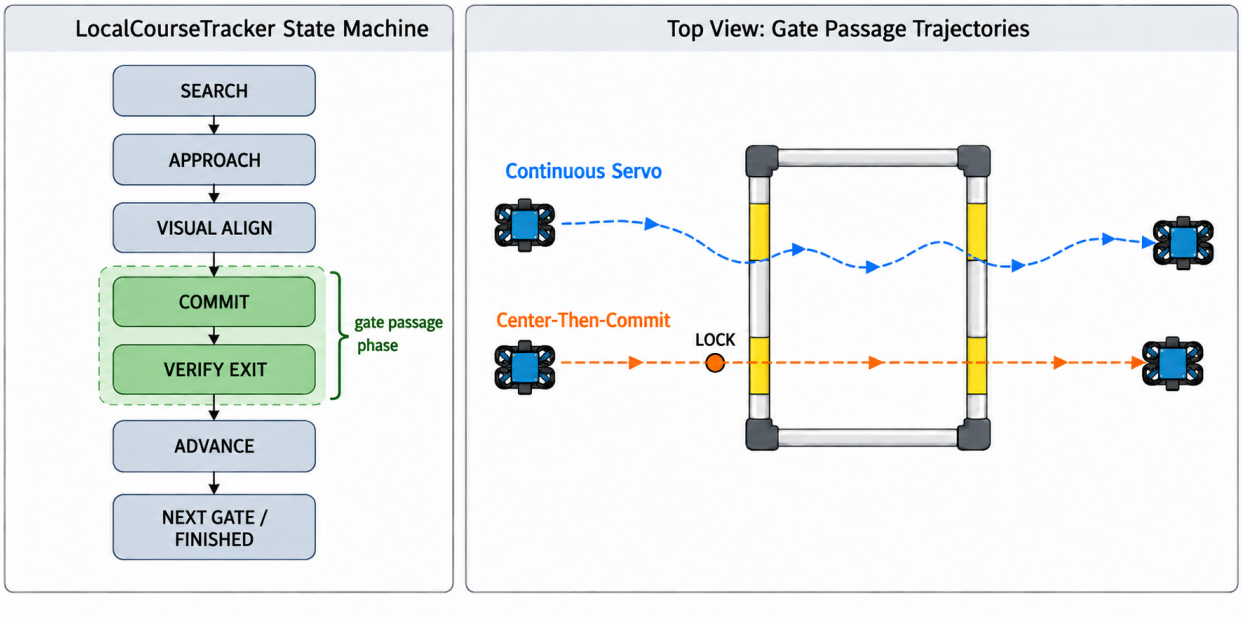

Figure 10: Conceptual LocalCourseTracker state machine and the two passage styles: continuous servoing and center-then-commit.

7. Independent Referee and the Information Boundary

The referee is the component that makes results comparable, and it is the component a participant must not be able to influence. This section formalizes the two-sided information boundary, the gate-crossing test, the referee state machine and arbitration order, the penalty model and the scoring rules, and it closes by treating the boundary as something to be measured rather than asserted.

7.1. The Two-Sided Boundary

The benchmark partitions the run state into two disjoint views. Writing x_t for the full simulator state at step t, the *controller view* of participant i is

$$o_{i,t} = \Phi_i(x_t) = (\, t_i^{\mathrm{loc}},\ \mathcal{Z}_{i,t},\ \mathcal{A}_{i,t},\ \mathcal{M}_{i,t}\,), \tag{15}$$

where t_i^{loc} is time since this participant's release, $\mathcal{Z}_{i,t}$ the allow-listed onboard measurements, $\mathcal{A}_{i,t}$ the set of acoustic packets delivered by the simulated channel at step t, and $\mathcal{M}_{i,t}$ the optional teammate inbox. The *evaluator view* is

$$e_t = \Psi(x_t) = (\, \{p_{i,t},\ \dot{p}_{i,t}\}_{i\in\mathcal{P}},\ \mathcal{G},\ \mathcal{O},\ \mathcal{B},\ \mathbf{v}_c\,), \tag{16}$$

comprising exact vehicle poses and velocities, the ordered gate geometry, the obstacle set, the world bounds and the true current field. The benchmark enforces

$$\Phi_i \perp \sigma\Big(\ell_t,\ G_{i,t}^{\mathrm{done}},\ \mathrm{status}_{i,t},\ P_{i,t}\Big) \quad \text{for all } i, t, \tag{17}$$

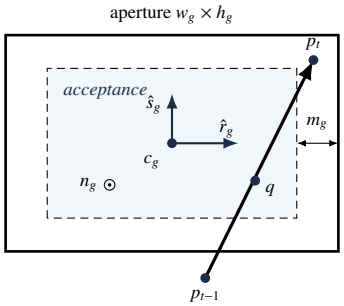

Figure 11: Gate-crossing geometry and the safety-margin-adjusted acceptance region.

that is, the observation map is independent of the referee's expected-gate index ℓ_t, of official completed-gate counts, of participant status and of accrued penalties. In the implementation this is a structural property rather than a convention: the observation builder's signature admits the track configuration, the arena, the adapter and the participant state, and does not admit a referee object, so referee-derived quantities are unreachable from the code path that constructs $o_{i,t}$.

Information flows in one direction only. The referee reads x_t and the controller's issued commands; the controller reads neither e_t nor any function of it. In particular, a controller is never told that a crossing was accepted, never told which gate is expected next, and never told that it has been penalized. Where the controller's own estimate of its progress disagrees with the referee's, the disagreement is logged and left standing (Section 7.7).

7.2. Gate Frame and Crossing Test

The referee reuses the right-handed gate frame $(\hat{n}, \hat{r}_g, \hat{s}_g)$ of (2), defined from the passage direction n_g with the vertical-normal fallback to the world $\hat{x}$ axis. For consecutive referee-side positions p_{t-1}, p_t, the signed distances to the gate plane are

$$d_{t-1} = \hat{n}^\top (p_{t-1} - c_g), \qquad d_t = \hat{n}^\top (p_t - c_g). \tag{18}$$

A candidate crossing requires a sign change in the *correct direction*,

$$d_{t-1} \le 0 \le d_t \ \wedge \ (p_t - p_{t-1})^\top \hat{n} > 0, \tag{19}$$

i.e. travel from the negative to the positive side along the normal. The intersection of the segment with the plane is

$$\lambda = \frac{-d_{t-1}}{d_t - d_{t-1}}, \quad q = p_{t-1} + \lambda\,(p_t - p_{t-1}), \quad \lambda \in [0, 1], \tag{20}$$

and the crossing is accepted only when q lies inside the aperture, shrunk by the configured safety margin m_g,

$$\left|\hat{r}_g^\top (q - c_g)\right| \le \tfrac{w_g}{2} - m_g, \qquad \left|\hat{s}_g^\top (q - c_g)\right| \le \tfrac{h_g}{2} - m_g. \tag{21}$$

Gates must be passed *in sequence*: the referee tests only the expected gate g_ℓ, advancing the index $\ell \leftarrow \ell + 1$ on a valid crossing and incrementing the lap when the sequence completes. Crossing the aperture of a *non-target* gate is a missed-gate event; a sign change in the wrong direction is logged but neither penalized nor counted as progress. Fig. 11 illustrates the test and Fig. 12 the principal automatic lifecycle.

Two design choices in this test are worth naming. Testing only the expected gate, rather than all gates, is what turns the course into an ordered task instead of a collection of independent hoops, and it is the reason the dominant learned-policy failure mode in Section 10.6 is a missed-gate termination rather than a low gate count. Shrinking the aperture by m_g before the containment test makes the pass criterion strictly conservative: a trajectory that clips the frame is not credited. The margin and the aperture dimensions were fixed before any experiment reported here and were never adjusted (requirement R4).

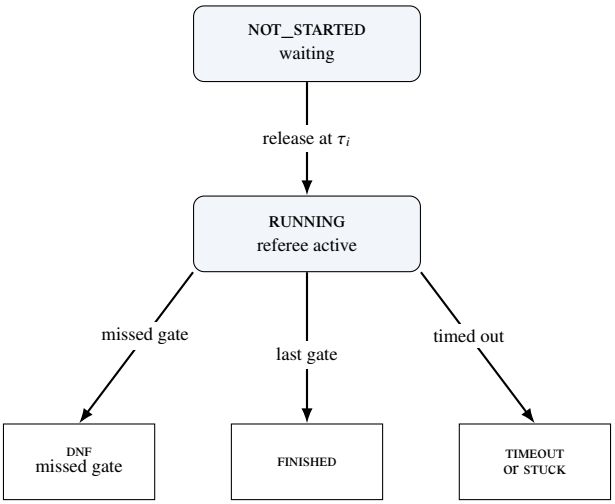

Figure 12: Simplified automatic participant lifecycle; manual-stop and controller-error terminals are omitted.

Table 8: Default referee events, penalties and terminal conditions.

Event	Penalty	Terminal
Gate, bar or generic collision	5.0 s each	no
Obstacle collision	5.0 s each[†]	no
Out of bounds	10.0 s each	no
Stuck (soft)	15.0 s per episode	no
Missed gate	N/A	yes (DNF)
Timeout	N/A	yes
Gate-timeout stuck	N/A	yes
Inter-vehicle proximity	0 or 5.0 s per pair	no (team-level)

[†]Per-obstacle value if specified, else the collision default. Collision, obstacle, out-of-bounds and inter-vehicle proximity charges use independent configured cooldowns (default 1.0 s); stuck uses one latch per episode and terminal events have no cooldown.

7.3. Referee State Machine and Arbitration

Each participant occupies one of the states NOT_STARTED, RUNNING, FINISHED, DNF, TIMEOUT, STUCK, DSQ, MANUAL_STOP or CONTROLLER_ERROR; all but the first two are terminal. A participant is released to RUNNING when the race clock reaches its start delay (Section 8). On each tick of a running participant, events are evaluated in a fixed priority order that defines the arbitration: (1) a controller error transitions to CONTROLLER_ERROR and returns; (2) out-of-bounds; (3) generic collision; (4) obstacle collision; (5) stuck accumulation; (6) duration timeout; (7) gate progression. Steps (2) to (5) accrue penalties without terminating; only (1), (6), a missed gate in (7) or an external gate-timeout produce a terminal state.

The order is not arbitrary. Placing controller errors first prevents a crashed participant from also accruing collision or stuck charges from its final frozen command. Placing gate progression last ensures that a tick in which the vehicle both crosses a gate and touches its frame is charged for the contact and credited for the crossing, rather than either event masking the other. To avoid double counting, an obstacle-collision tick suppresses the generic collision flag. Repeated collision, obstacle and out-of-bounds charges use independent cooldowns; inter-vehicle proximity uses its own team-level cooldown; stuck is latched once per episode; and terminal events are not cooldown-limited.

7.4. Penalties and Termination

Penalties accumulate into a per-participant total P_i; the applied defaults are listed in Table 8. The non-trivial conditions are formalized as follows. The instantaneous speed is $s_t = \|p_t - p_{t-1}\|/\Delta t$, and a stuck accumulator

integrates low-speed time,

$$T^{\text{stuck}} \leftarrow \begin{cases} T^{\text{stuck}} + \Delta t, & s_t < v_{\text{stuck}} \\ 0, & \text{otherwise,} \end{cases} \tag{22}$$

with a soft penalty charged once per continuous low-speed episode when $T^{\text{stuck}} \geq T^{\text{stuck}}_{\text{max}}$ and $v_{\text{stuck}} = 0.02\,\text{m/s}$. The configured $T^{\text{stuck}}_{\text{max}}$ is 45 s on Horseshoe Bay and 65 s on Vertical Serpent and Mixed Endurance. An out-of-bounds event is $p_t \notin \mathcal{B}$. A duration timeout, disabled by default, fires when $t - t_{\text{green}} > T_{\text{max}}$.

A missed gate sets DNF when `missed_gate_dnf` is enabled, which is the default, but carries no time penalty. The combination is deliberate: making a disqualifying error terminal but free of accumulated time ensures that losing the course is never cheaper than completing a slow but legal lap, which would otherwise be an exploitable scoring artefact.

7.5. Single-Vehicle Score and Ranking

For a finished participant the official time T^{official}_i is measured between the first and last gate (or green-to-finish, according to the timing mode) and the score is the penalized time

$$T^{\text{pen}}_i = T^{\text{official}}_i + P_i. \tag{23}$$

Ranking places finished participants before unfinished ones and is defined by the ascending lexicographic key

$$\kappa_i = \begin{cases} (0, T^{\text{pen}}_i, 0, 0, 0, \text{id}_i), & f_i = 1, \\ (1, -G^{\text{done}}_i, n^{\text{coll}}_i, P_i, \text{dist}_i, \text{id}_i), & f_i = 0, \end{cases} \tag{24}$$

where $f_i = 1$ denotes a finished vehicle, G^{done}_i the number of completed gates, n^{coll}_i the collision count and dist_i the distance to the next gate; the participant identifier is the deterministic final tie-break. Finished vehicles are therefore ordered by penalized time, and unfinished vehicles by progress, then by fewer collisions, fewer penalties and proximity to the next gate. This ordering preserves the information in partial runs instead of discarding them, which matters for the learned controllers of Section 9, most of whose official-circuit episodes are partial.

7.6. Event Logging

Every run emits a line-delimited event log and a machine-readable summary (requirement R5). Events include `gate_passed`, `wrong_direction`, `collision`, `obstacle_collision`, `out_of_bounds`, `stuck`, `race_finish` and `dnf`, each timestamped and attributed to a participant. For each participant, the summary records the status, the official and penalized times, the completed gates, the collision count, out-of-bounds events, missed gates and stuck events, together with the final ranking. In fleet mode it also carries the team summary of Section 8. Every table in Section 10 is an aggregation over these files.

7.7. Measuring the Boundary

An information boundary that is only asserted is a design claim; one that is measured is evidence. Because the controller maintains its own course estimate (Section 6.3) and the referee maintains the official one, the two can be compared offline after every run, and the framework records that comparison automatically. We define a *false local advancement* as a local increment that precedes its same-ordinal referee crossing by more than one control step, or that has no corresponding referee crossing at all, and a *missed local advancement* as a referee crossing with no same-ordinal local increment in the run. The advancement delay is the controller-local advancement time minus the referee crossing time, so a positive delay means the controller learned of its own progress after the referee recorded it.

Table 9 reports this comparison over the frozen 78-run matrix. Across 1474 referee advancements the controller produced 1472 local advancements, of which 1467 matched the referee. The 11 false and 7 missed local advancements are 0.75% and 0.47% of the total. The controller consistently lags: the median delay is 0.858 s and the 95th percentile 1.353 s, which is the price of requiring camera, acoustic and DVL evidence to agree before advancing. No run recorded an event-consistency error, no finish order was inverted across 61 pairwise comparisons, and in no case did a controller declare itself finished before the referee agreed.

Table 9: Controller-local course progression against independent referee progression, over the frozen 78-run matrix. A positive delay means the controller confirmed its own passage *after* the referee recorded the crossing. No value in this table is fed back into control.

	Gate advancements			Mismatches		Local delay (s)	
Circuit	referee	local	matched	false	missed	median	p95
Horseshoe Bay	1120	1120	1116	10	4	0.924	1.353
Vertical Serpent	159	160	159	1	0	0.627	1.155
Mixed Endurance	195	192	192	0	3	0.462	1.287
All	**1474**	**1472**	**1467**	**11**	**7**	**0.858**	**1.353**

A *false* local advancement is a local increment that precedes its same-ordinal referee crossing by more than one control step, or that has no corresponding referee crossing; a *missed* local advancement is a referee crossing with no same-ordinal local increment. Across all runs there were 0 event-consistency errors, 0 finish-order inversions over 61 pairwise comparisons, and 0 cases of a controller declaring itself finished before the referee agreed; the referee recorded the finish before the local tracker in 103 cases. The mean delay is 0.722 s with a sample standard deviation of 2.83 s; the spread is dominated by a single false advancement at −90.8 s, which is retained rather than filtered.

Two conclusions follow, and they are the reason we regard this table as central rather than incidental. First, the boundary is real: the controller's estimate is a genuinely separate quantity that is sometimes wrong, in both directions, and nothing in the framework repairs it. Second, the boundary is not *prohibitive*: an onboard-only controller can track an ordered course closely enough that its own progress estimate is correct in more than 99% of advancements, so requiring onboard-only operation does not make the task unmeasurable. The single negative delay of −90.8 s in the raw distribution is a false local advancement in which the controller advanced its expected beacon long before the referee accepted the corresponding crossing; it is retained in the audit rather than filtered, which is why the delay standard deviation (2.83 s) far exceeds its interquartile spread.

8. Fleet Execution, Coordination and Team-Level Evaluation

Fleet mode runs several vehicles as a single cooperative team through the same circuit, so the benchmark measures how quickly and how safely the team completes the course rather than pitting the vehicles against each other (requirement R7). The benchmark models one team: all vehicles share a single simulated world, each controller keeps an independent local course estimate, and the referee keeps one scoring state per vehicle. Enabling fleet mode does not alter single-vehicle behaviour (R6), so a fleet result and a single-vehicle result remain comparable.

8.1. Staggered Release and Spawn Geometry

Each participant i is assigned a release delay $\tau_i = (i-1)\,\Delta\tau$ for a start gap $\Delta\tau$, so vehicles enter the course at $0, \Delta\tau, 2\Delta\tau, \ldots$ A waiting vehicle receives a zero command, its controller `step` is *not* called, and neither stuck nor gate-timeout timers accumulate. At release the referee logs a `participant_released` event and resets the vehicle's progress timers, so a delayed start is never misclassified as stuck or timed out. Spawn poses are laterally separated about the base spawn to reduce launch interference: using zero-based $k = 0, \ldots, N-1$, the k-th vehicle is offset by

$$o_k = (-1)^k \left\lceil \frac{k}{2} \right\rceil s \tag{25}$$

along the right axis $(-\sin\psi_0, \cos\psi_0)$ of the base yaw ψ_0, giving the alternating pattern $0, -s, +s, -2s, +2s, \ldots$ for spacing s; a candidate that would fall outside the world bounds $\mathcal{B}$ is scaled inward until admissible.

8.2. Inter-Vehicle Proximity Diagnostics

Inter-vehicle proximity events are detected on the referee side and are never exposed to controllers, which is a direct consequence of (17): a vehicle that could read the referee's proximity flag would be receiving privileged information about a teammate's exact position. For two running vehicles a, b the detector applies a separable cylinder test

$$\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2} \le r_{xy} \;\wedge\; |z_a - z_b| \le r_z, \tag{26}$$

with hysteresis — a pair is released only when the horizontal distance exceeds a release threshold r_{rel} or the vertical distance exceeds r_z — and a refractory cooldown to debounce sustained proximity. A vehicle that wants to avoid its teammates must therefore infer their presence from its own sensing and from messages they choose to send.

8.3. Inter-Vehicle Acoustic Communication

Because the fleet is a single cooperative team, Marine Race Arena provides an optional channel so that vehicles can share information while running the course. It is off by default and changes nothing for single-vehicle runs (R6). A controller transmits by returning a small `message` payload alongside its command and receives an `inbox` of teammates' messages in its observation; what a message contains and how it is used are entirely the participant's responsibility, so the framework provides only the transport.

To stay faithful to the medium rather than model a perfect radio link, the channel reproduces the constraints of the underwater acoustic channel discussed in Section 2. A message broadcast by vehicle i reaches a teammate j only if the two are within a maximum range $R_{\max}$, after a range-dependent delay

$$\tau_{ij} = \frac{d_{ij}}{c} + \tau_{\text{proc}}, \tag{27}$$

where d_{ij} is the inter-vehicle distance, c the speed of sound and τ_{proc} a fixed processing delay. Each delivery is dropped with probability p_{loss}. Payloads are capped to a small size to model the low bandwidth, and a vehicle may not transmit faster than a configured per-sender interval. Packet loss is drawn from a seeded generator, so a run remains reproducible.

We are explicit about the status of this component: it is an acoustic-inspired transport, not a characterized channel model. It provides the substrate for the coordination policy below and is exercised as part of that validation, but its packet loss, maximum range, latency and freshness window are not independently swept, and no claim about acoustic-communication performance is made in this paper.

8.4. Distributed Leader–Follower Coordination

Motivation. A staggered team of *identical* controllers stays separated in time, because each vehicle takes roughly as long as the one ahead. A *heterogeneous* team does not: a faster follower released behind a slower leader closes the gap, and the two then converge on the same aperture centre at the same gate, because both are servoing to the same visual target. The resulting contacts are a property of the convoy, not of either controller in isolation, and they are severe — Section 10.5 reports a mean of 127.3 gate and world collisions per run for an uncoordinated three-vehicle team at an 8 s gap.

Module design. We provide a coordination controller that maintains a spaced convoy using only legal information. It *wraps* rather than replaces a gate-passing controller that exposes a `LocalCourseTracker`, which keeps coordination and gate navigation separate concerns; both official camera-assisted controllers satisfy this interface. Each vehicle is assigned a predecessor from the static release order carried in its legal mission information; the frontmost vehicle has no predecessor and races freely. Within the channel's transmit-rate limit, every vehicle periodically broadcasts exactly three fields — `local_beacon_index`, `local_lap` and `local_status` — read from its own base controller's tracker. The receiver treats these as estimates, retains the most recently received predecessor report and ignores it after a 2.5 s freshness window.

Let $\hat{g}_i$ denote cumulative locally estimated progress reconstructed from the reported lap and beacon index. A follower yields while its predecessor is fewer than Δg locally estimated gates ahead,

$$\text{yield}_j \iff \hat{g}_{\text{pred}(j)} - \hat{g}_j < \Delta g, \tag{28}$$

and otherwise runs its wrapped base controller unchanged. While yielding, the wrapper still updates the base tracker from onboard observations but commands zero motion. A predecessor locally reported as finished, a missing report or a report older than the freshness window all stop blocking. With the channel disabled no report is delivered and the wrapper reduces exactly to the uncoordinated base behaviour.

Technical advantage and its cost. Every input to (28) is legal: the vehicle's own tracker, a static predecessor assignment and a controller-authored message that itself contains only the sender's own estimate. Referee progress can disagree with either local estimate but never changes a coordination decision. The cost is that a deliberate wait is indistinguishable, to the referee, from being stuck: yielding can trigger the independent stuck penalty of (22), and Section 10.5 shows this happening for the more conservative two-gate margin. We regard that as correct behaviour for a benchmark — a coordination policy that buys safety by standing still should pay for the time it spends standing still — and it is one reason the recommended default margin is one gate rather than two.

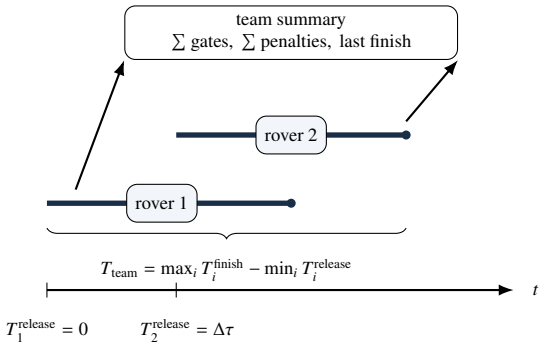

Figure 13: Team-level scoring for a staggered two-rover fleet.

8.5. Team-Level Score

Only the `team_summary` is the official fleet result; per-vehicle rows are diagnostics. For a team $\mathcal{P}$ of N vehicles with G_{exp} expected gates each,

$$G_{\text{team}}^{\text{exp}} = N\, G_{\text{exp}}, \qquad G_{\text{team}}^{\text{done}} = \sum_{i \in \mathcal{P}} G_i^{\text{done}}. \tag{29}$$

The team finishes only if every vehicle finishes,

$$\text{finished}_{\text{team}} = \bigwedge_{i \in \mathcal{P}} \text{finished}_i, \tag{30}$$

in which case the team elapsed time spans the first release to the last finish,

$$T_{\text{team}} = \max_i T_i^{\text{finish}} - \min_i T_i^{\text{release}}, \tag{31}$$

and the penalized team score aggregates every per-vehicle penalty together with the team-level inter-vehicle term,

$$T_{\text{team}}^{\text{pen}} = T_{\text{team}} + \sum_{i \in \mathcal{P}} P_i + P_{\text{ivc}}, \tag{32}$$

where P_i is the per-vehicle penalty total of (23) and P_{ivc} counts each inter-vehicle proximity event once per pair rather than once per vehicle. The conjunction in (30) is what makes the score a *team* score: a fast leader cannot compensate for a follower that never finishes, so a coordination policy is rewarded for bringing the whole convoy home. Fig. 13 illustrates the aggregation.

Figure 14 summarizes the multi-vehicle information flow and the independent team referee.

9. Learning-Based Extension

Learning is included as a supporting demonstration that a learned controller can enter Marine Race Arena through the same participant interface as the reference controllers. It does not replace the benchmark's central contribution: the official observation boundary, ordered-gate referee and scoring protocol remain unchanged.

Figure 15 places the recurrent policy in that interface and shows the curriculum progression used before the frozen validation.

9.1. Final Gen-2 Observation Contract

The final experiment uses the committed `onboard_local_transition_27d_v1` contract. Its 27 bounded features are ordered as follows: beacon presence, bearing sine and cosine, elevation and range; visual centre coordinates and area; depth; DVL surge, sway and heave plus presence; IMU yaw rate; the previous surge, sway, heave and yaw actions; five short-term rates for visual centre, acoustic bearing, elevation and range; a recent-target-change flag; and

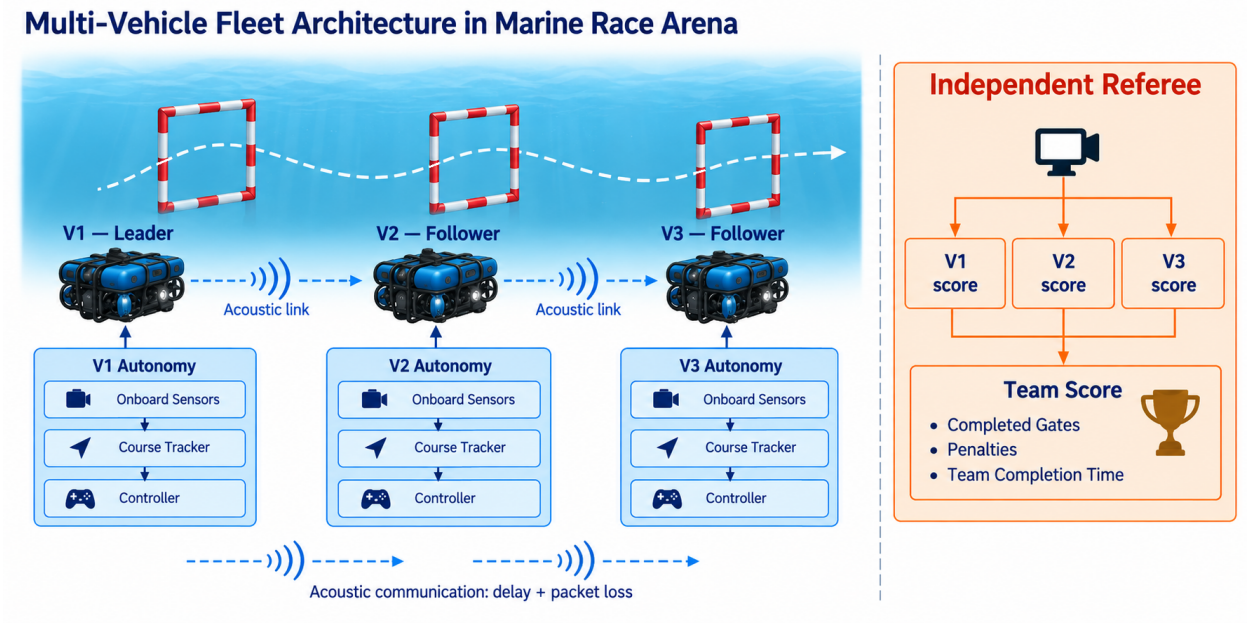

Figure 14: Conceptual fleet architecture with leader–follower communication, independent vehicle autonomy and an independent referee producing team score.

gate-orientation presence with yaw sine and cosine. Missing values follow the explicit contract: missing visual centre and area are zero, invalid rates are zero, and an unavailable orientation contributes zero mask and trigonometric components. The DVL presence bit is retained.

The encoding contains no gate coordinates, global pose, referee progress, future geometry or reward. It is therefore a local transition representation, not a privileged course-state representation. The four continuous body-frame actions are the same `surge`, `sway`, `heave` and `yaw` channels in $[-1, 1]$ used by the rule controllers; framework clamping and depth limits are shared.

9.2. Recurrent PPO and Training Configuration

The final policy uses Stable-Baselines3 Contrib RecurrentPPO with the committed Gen-2 recurrent-LSTM architecture:

`gen2_recurrent_lstm_27d_v1`

a frozen normalization buffer, a $27 \to 256 \to 128$ tanh encoder, separate one-layer actor and critic LSTMs with hidden size 128, 128-unit policy/value heads, and a four-action Gaussian head. The recorded environment is CPU-based; the dependency lock records Stable-Baselines3 and SB3-Contrib 2.7.1 with PyTorch 2.8.0.

The final direct-speed campaign starts from the committed Gen-2 initialization checkpoint and uses full copies of the three official circuits, three workers, current-free conditions and the recorded fog configuration. It uses no track fragments or synthetic circuits in the reported campaign. PPO settings are learning rate 6×10^{-6}, clip range 0.07, target KL 0.008, two epochs, 128 batch size, 256 rollout steps, zero entropy coefficient and gradient norm limit 0.5. The direct-speed reward records gate crossing (+20), completion (+50), time (−0.01 per step), terminal failure (−55), collision (−20), directness (+0.025) and aligned speed (+0.015); energy and jerk terms are zero in this campaign.

The preceding Gen-2 data, behavioural-cloning and DAgger artifacts are retained as initialization and audit history. They are not pooled with the final PPO validation. The official circuits were available to earlier diagnostic runs, so the present circuit results are retrospective validation of the frozen integration, not an unseen-track generalization claim.

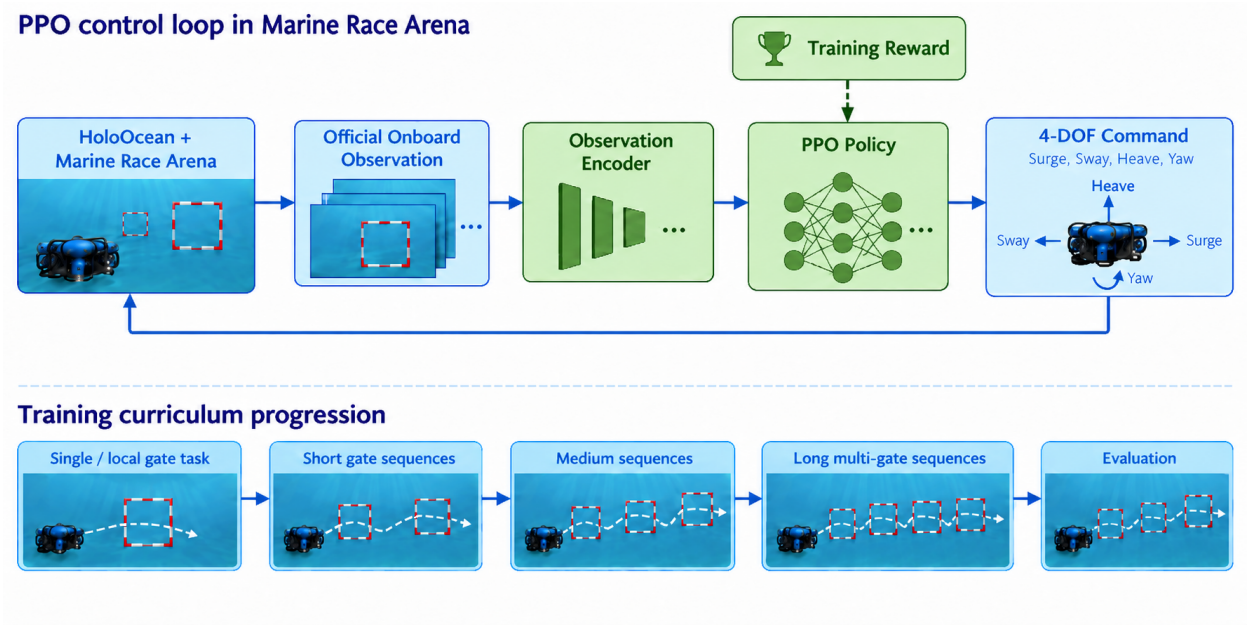

Figure 15: Conceptual Gen-2 recurrent-policy loop and curriculum progression from local transitions to longer multi-gate evaluation.

9.3. Checkpoint Selection and Matched Evaluation

Checkpoint selection is recorded separately from training in `training_decision_log.json`. The selected campaign is `ppo_27d_direct_speed_campaign_C_20260909_retry1`; its first registered milestone produced the selected `checkpoint_050000.zip` at 50,688 actual environment steps. Its SHA-256 is recorded alongside the checkpoint and validation manifest. Later 100k diagnostics were not selected because they degraded or partially failed robust evaluation.

The final validation contains three valid episodes per official circuit at the learning control rate. Horseshoe Bay finished 3/3 with no collisions or out-of-bounds events; Vertical Serpent finished 3/3 with one collision event; Mixed Endurance finished 3/3 with no collisions or out-of-bounds events. The results are descriptive ($n = 3$ per circuit), not a statistical superiority claim. The current-free rule demonstration is a separate 10-Hz artifact and finishes all three circuits in 9/9 runs without collision, out-of-bounds or wrong-direction events; its completion rate is a useful interface reference, but its per-run times are not pooled with the PPO table.

9.4. Information-Boundary Interpretation

The learned policy receives the same onboard-only stream and is scored by the same independent referee as every other participant. Consequently, the main learning result is not that PPO solves underwater racing. It is that the benchmark exposes a meaningful distinction between local gate-transition control and reliable ordered-sequence completion: the selected policy completed all nine validation episodes, while the surrounding audit keeps the contract, checkpoint hash and episode-level evidence independently inspectable.

10. Experimental Evaluation

This evaluation does not seek an optimal underwater racing controller. It asks whether the benchmark, the observation contract, the referee and the team scoring behave as specified, and whether they expose differences between autonomy strategies that a coarser protocol would hide. We organize the experiments around five research questions rather than chronologically, and we report failures of the reference controllers and of the learned policies as results rather than as caveats.

RQ1 Can Marine Race Arena support fully onboard autonomous completion across heterogeneous underwater racing circuits?

RQ2 Does the benchmark expose meaningful differences between autonomous control strategies?

RQ3 How do environmental currents affect benchmark performance?

RQ4 Can the same evaluation framework support multi-vehicle and team-level racing?

RQ5 Can learning-based controllers be integrated and evaluated under the same observation and referee contract?

10.1. Protocol, Metrics and Provenance

Three independent bodies of evidence are reported, and they are never pooled.

The systematic benchmark matrix. 78 runs generated with the native HoloOcean simulator backend under a frozen source-tree fingerprint recorded in the artifact manifest. It covers both reference controllers on the three clean circuits and under the `medium` and `strong` Horseshoe Bay currents, homogeneous two-vehicle fleets, and matched three-vehicle coordination trials. Clean, current and fleet cases use five seeds; coordination uses three seeds under simultaneous and staggered release. Control runs at $\Delta t = 0.033$ s (30 Hz). This matrix answers RQ2, RQ3 and RQ4, and supplies the boundary measurement of Section 7.7.

The current-free completion demonstration. A focused nine-run demonstration that the reference controller completes each official circuit end to end with currents explicitly disabled and verified zero in the manifest, at $\Delta t = 0.1$ s (10 Hz). This answers RQ1.

The final learned-controller validation. The selected Gen-2 27-D RecurrentPPO checkpoint was evaluated in three valid episodes on each official circuit at $\Delta t = 0.1$ s. This answers RQ5 as an integration and diagnostic question; it is not pooled with the 78-run matrix.

All performance trials use the reference native HoloOcean simulator backend and a BlueROV2-class vehicle, official onboard-only observations, no current compensation and no obstacles, with the standard gate apertures and referee margin unchanged throughout (R4). Separate setup checks confirm that both named current profiles and seeded static obstacles instantiate on every circuit; these are construction checks, not obstacle-avoidance trials, and no obstacle result is claimed.

The reported metrics are completion, completed gates, official and penalized time, collisions, out-of-bounds events, wrong-direction crossings and stuck events. Fleet and coordination trials add inter-vehicle proximity events and team times. Time statistics use finished runs only and are undefined where nothing finished; gate and event means use all seeds. Dispersions are sample standard deviations (ddof = 1). Proportions carry Wilson 95% intervals where sample sizes are small enough for the normal approximation to mislead. Paired comparisons use two-sided sign tests on matched seeds.

Every run records the experiment-generation commit, source or model hash, HoloOcean version, track hash, requested and actual adapter, fallback status, current profile, timestep and seed. Runs that used a non-HoloOcean adapter or permitted the engine-free fallback do not appear in the tables.

10.2. RQ1: Onboard-Only Completion Across Heterogeneous Circuits

The first question a racing benchmark must answer about itself is whether its task is solvable at all under its own restrictions. If no controller can complete a circuit using only onboard sensing, the benchmark measures the difficulty of an impossible problem rather than the quality of autonomy.

Table 10 answers this affirmatively and without qualification for the current-free setting. The Center-then-Commit controller completed all three official circuits in every one of nine runs: 3/3 on Horseshoe Bay (12 gates), 3/3 on Vertical Serpent (17 gates) and 3/3 on Mixed Endurance (22 gates), for 51 of 51 gates in total. Across those nine runs the referee recorded zero gate, bar or world collisions, zero out-of-bounds events and zero wrong-direction crossings, and penalized time equalled official time in every run because no penalty was ever charged. Every run manifest independently confirms the conditions under which this was achieved: the native HoloOcean simulator backend was

Table 10: Current-free autonomous completion of the three official circuits by the Center-then-Commit reference controller. Native HoloOcean simulator backend, engine-free fallback disabled, currents verified zero in the manifest, no privileged controller input, $\Delta t = 0.1$ s, seeds 1800–1802.

Circuit	Gates	Runs	Completed	Mean gates	Official time (s)	Coll.	OOB / WD
Horseshoe Bay	12	3	3/3	12.0/12	236.0 ± 6.9	0	0 / 0
Vertical Serpent	17	3	3/3	17.0/17	308.6 ± 6.5	0	0 / 0
Mixed Endurance	22	3	3/3	22.0/22	480.4 ± 15.7	0	0 / 0
Total	**51**	**9**	**9/9**	**51.0/51**	—	**0**	**0 / 0**

Times are mean ± sample standard deviation (ddof= 1) over the three runs; penalized time equals official time in every run because no penalty was charged. *Coll.*: gate, bar and world collision events. *OOB / WD*: out-of-bounds events and wrong-direction crossings. Every run manifest records `adapter_actual` = `holoocean`, `fallback_allowed` = false, `current_profile` = `none` and `currents_actual` = $[0, 0, 0]$, together with the track SHA-256 and the source commit. Mixed Endurance is a `current_gate` task whose validation requires a current, so a current-free run additionally overrides the validation mode to `clean_gate`; the track geometry, gate order, laps and referee configuration are unchanged. These runs use a 10 Hz control rate and are therefore not directly comparable in time with Table 11, which uses 30 Hz.

Table 11: Systematic clean-track evaluation of the two reference controllers over five seeds per case, from the frozen 78-run HoloOcean matrix ($\Delta t = 0.033$ s, 30 Hz control). Times use finished runs only; gate and event means use all five seeds.

Track	Controller	Seeds	Finished	Gates ↑	Official time (s) ↓	Collisions ↓
Horseshoe Bay	Continuous Servo	5	5/5	12.0/12	194.5 ± 4.8	0.0
	Center-then-Commit	5	5/5	12.0/12	197.7 ± 7.1	0.0
Vertical Serpent	Continuous Servo	5	**5/5**	**17.0/17**	236.9 ± 6.4	0.0
	Center-then-Commit	5	4/5	14.8/17	241.1 ± 1.8	0.0
Mixed Endurance	Continuous Servo	5	2/5	17.0/22	394.7 ± 3.9	0.0
	Center-then-Commit	5	**5/5**	**22.0/22**	410.7 ± 8.8	0.0

Times are mean ± sample standard deviation (ddof= 1) over finished runs; gates and collisions are five-seed means, and collisions combine gate and world events. No clean run recorded an out-of-bounds or stuck event, and penalized time equals official time throughout. Bold marks the better controller where the two differ in completion. Regenerated from the raw per-run summaries by `article/regenerate_tables.py`.

used, the engine-free fallback was disabled, the requested and measured current vector was $[0, 0, 0]$, and the controller received no privileged input.

Completion time scales with course length roughly as expected from the track geometry: 236.0 ± 6.9 s over 93.8 m, 308.6 ± 6.5 s over 118.3 m and 480.4 ± 15.7 s over 206.3 m, i.e. approximately 0.40, 0.38 and 0.43 m/s of course progress. The consistency of that rate across three structurally different circuits — planar, vertically serpentine and long mixed — indicates that the limiting factor is the controller's gate-by-gate acquire–align–commit cycle rather than any circuit-specific difficulty.

We are deliberate about the scope of this result and about its relationship to the systematic matrix. This is a *focused completion demonstration*: three seeds per circuit, one controller, no disturbance, at a 10 Hz control rate. It establishes that the task is achievable end to end under the full observation contract, and it does so with a per-run manifest that makes the claim auditable. It is not a performance characterization, and it does not supersede the broader evaluation of Sections 10.3–10.5, which uses five seeds, both controllers, three disturbance conditions and a 30 Hz control rate. The two are complementary and are reported separately for that reason; in particular, the times in Table 10 are systematically longer than those in Table 11 because the control rate differs by a factor of three, and the two must not be compared.

10.3. RQ2: Do Control Strategies Separate?

A benchmark that ranks every reasonable controller identically is not measuring anything. The two reference controllers of Section 6 are an unusually clean test of this, because they differ in exactly one respect — what happens during COMMIT and VERIFY_EXIT — while sharing observations, guidance, tracker and confirmation logic.

Table 11 and Fig. 16 show that the answer depends on the circuit, and that neither controller dominates. On the short, largely planar Horseshoe Bay circuit the two are effectively tied: both complete all 12 gates in all five seeds with no collision, out-of-bounds or stuck event, and the staged controller is 3.2 s slower (197.7 ± 7.1 s versus 194.5 ± 4.8 s),

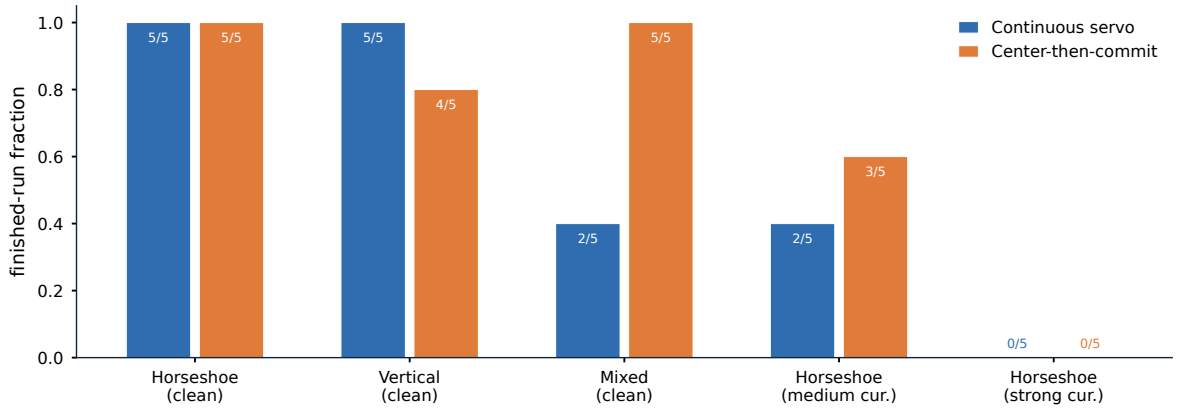

Figure 16: Finished-run fraction of the two reference controllers over five seeds per condition, from the frozen 78-run matrix; each label reports finished runs out of five. Neither controller dominates: they tie on the short planar circuit, Continuous Servo leads on Vertical Serpent, and Center-then-Commit leads on Mixed Endurance and under the medium current.

a 1.7% penalty with no measured safety benefit. Constant re-centring is cheap here, and paying nothing for it is the right trade.

The controllers separate on the harder circuits, and they separate in opposite directions. On Vertical Serpent, Continuous Servo is more reliable, finishing $5/5$ against $4/5$ and completing 17.0 against 14.8 mean gates. On Mixed Endurance, which is more than twice as long, the ordering reverses far more sharply: Center-then-Commit finishes all five seeds and completes every gate ($22.0/22$), while Continuous Servo finishes only $2/5$ and averages $17.0/22$.

The mechanism is consistent with the design argument of Section 6.5. Continuous visual correction is harmless when a passage is short and the approach is well conditioned, and it becomes a liability when partial near-field contours make the image centroid move quickly at exactly the moment the aperture margin is smallest. A long course compounds this: each passage is an independent opportunity for a late correction to fail, so a small per-gate risk becomes a large per-course risk. That Continuous Servo nonetheless wins on Vertical Serpent shows the trade is not one-directional — repeated vertical transitions appear to benefit from continuous re-centring more than they suffer from late corrections — which is precisely the kind of condition-dependent result a single-circuit benchmark would have concealed. The comparison is limited to the three official circuits, two current profiles and the evaluated seeds.

10.4. RQ3: Behaviour Under Environmental Currents

Performance degrades sharply and non-uniformly as current strength increases (Table 12). Under the `medium` Horseshoe Bay profile, Continuous Servo finishes $2/5$ runs, averages $8.4/12$ completed gates and 43.4 gate and world collisions per run, with finished-only official and penalized times of 337.9 ± 19.2 s and 692.9 ± 75.7 s. Center-then-Commit finishes $3/5$, averages $8.8/12$ gates and 25.2 collisions, with finished-only times of 217.3 ± 1.5 s and 223.9 ± 1.8 s. Neither controller finishes a single `strong` run: Continuous Servo averages $3.0/12$ gates with no gate or world collision, and Center-then-Commit $3.2/12$ gates with 0.8.

Two features of these numbers require comment, because both are easy to misread.

First, the collision counts are *non-monotonic* in current strength: the medium profile records many collisions (43.4 and 25.2) while the strong profile records almost none (0.0 and 0.8). This is not evidence of better control under the stronger disturbance. Under `strong` the vehicles complete only 3.0–3.2 of 12 gates and stall early, so they rarely reach the gate-dense region of the circuit where contacts accumulate. Fewer collisions here reflect less progress, not more skill, and a benchmark that reported collision counts without completion would invert the ranking.

Second, the `strong` profile is best read as a saturation regime rather than a graded difficulty. Its constant set-flow component alone is $[0.75, 1.05, 0]$ m/s, a magnitude of 1.29 m/s, about $2.7\times$ the mean course speed the same vehicle sustains on clean Horseshoe Bay (93.8 m in 194.5 s, i.e. ≈ 0.48 m/s). No controller restricted to this vehicle's actuation can make sustained forward progress against it, so the condition functions as a stress case that confirms the disturbance is physically coupled rather than as an informative comparison point. The `medium` profile (0.65 m/s

Table 12: Robustness to environmental currents on Horseshoe Bay, five seeds per case, from the frozen 78-run matrix ($\Delta t = 0.033$ s). The clean row is repeated from Table 11 as the zero-disturbance reference.

Profile	Controller	Seeds	Finished	Gates ↑	Official (s)	Penalized (s)	Collisions ↓
None	Continuous Servo	5	5/5	12.0/12	194.5 ± 4.8	194.5 ± 4.8	0.0
	Center-then-Commit	5	5/5	12.0/12	197.7 ± 7.1	197.7 ± 7.1	0.0
`medium`	Continuous Servo	5	2/5	8.4/12	337.9 ± 19.2	692.9 ± 75.7	43.4
	Center-then-Commit	5	**3/5**	**8.8/12**	217.3 ± 1.5	223.9 ± 1.8	**25.2**
`strong`	Continuous Servo	5	0/5	3.0/12	—	—	0.0
	Center-then-Commit	5	0/5	3.2/12	—	—	0.8

Times are mean ± sample standard deviation over finished runs only and are undefined where no run finished. Gates and collisions are five-seed means, and collisions combine gate and world events. The `medium` profile has a constant set-flow component of 0.65 m/s and `strong` of 1.29 m/s, against a mean clean course speed of ≈0.48 m/s for the same vehicle on this circuit. The near-zero collision counts under `strong` reflect less progress, not better control: both controllers stall after about three gates and rarely reach the gate-dense part of the course. Run metadata confirms physical current coupling in every run.

constant component) is the informative regime, and there the staged controller is clearly the more robust: more completions, roughly half the collisions, and finished runs 120.6 s faster.

The headline finding for benchmark design is the gap between Table 11 and Table 12. Both controllers score a perfect 5/5 with zero events on clean Horseshoe Bay, and drop to 2/5 and 3/5 with tens of collisions on the same circuit under a moderate current. Clean-track success is therefore not a proxy for robustness, and any evaluation protocol that reports only clean completion will rank controllers on a dimension that disappears the moment the water moves. The quantitative current sweep is limited to Horseshoe Bay (Section 12).

10.5. RQ4: Multi-Vehicle and Team-Level Evaluation

The upper block of Table 13 validates that team execution works. With either controller, a homogeneous two-vehicle fleet on clean Horseshoe Bay at a 90 s release gap finishes 24/24 team gates in all five seeds, with no gate or world collision, no inter-vehicle proximity event, no out-of-bounds and no stuck event. Mean team elapsed and penalized times coincide at 303.1 ± 3.2 s for Continuous Servo and 308.0 ± 5.8 s for Center-then-Commit. This is deliberately a low-contact case: it exercises multi-vehicle spawning, staggered release, independent per-vehicle referee state, ranking and team aggregation without the vehicles interacting, and it confirms requirement R6, since the per-vehicle behaviour matches the single-vehicle case.

The interacting case is the interesting one. A heterogeneous three-vehicle convoy — a Continuous Servo leader followed by two Center-then-Commit vehicles — converges on the same apertures, and the consequences are severe. At an 8 s gap the uncoordinated convoy finishes only 2/3 runs, averages 34.3/36 team gates, 127.3 gate and world collisions and 2.0 inter-vehicle proximity events per run. At simultaneous release it still finishes 3/3, but averages 9.3 collisions and 2.3 proximity events.

The distributed leader–follower policy of (28) removes this entirely. At both gaps, LF(1) finishes 3/3 with 36.0/36 team gates and zero gate, world, proximity, out-of-bounds and stuck events, at 266.0 ± 8.3 s (gap 8 s) and 262.3 ± 7.7 s (gap 0 s). The more conservative LF(2) also eliminates contact but is consistently slower (285.7±3.8 s and 288.3±1.3 s) and, at simultaneous release, its longer hold trips the independent stuck timer once per run, raising its penalized team time to 303.3 ± 1.3 s. A larger nominal gate separation therefore bought no additional safety in these conditions while costing both time and a penalty, which is why LF(1) is the recommended default and LF(2) is retained only as a conservative comparison.

Two properties of this result matter beyond the numbers. The coordination policy consumes only the vehicle's own tracker state, a static predecessor assignment and teammate-authored messages delivered over a channel with range-dependent latency and loss; it never reads referee progress or a teammate's true position, so the improvement is achieved inside the same information boundary that constrains everything else. And the cost of coordination is visible in the score rather than hidden: LF(2)'s stuck penalty is charged by the same independent referee that charges a collision, so a policy that buys safety by waiting pays for the waiting. This evidence is limited to one clean circuit, three vehicles, two release gaps and three seeds.

Table 13: Multi-vehicle evaluation on clean Horseshoe Bay, from the frozen 78-run matrix ($\Delta t = 0.033$ s). The upper block validates team execution with a homogeneous two-vehicle fleet at a 90 s release gap (five seeds); the lower block studies a heterogeneous three-vehicle convoy under two release gaps (three seeds per condition).

Setting	Policy	Seeds	Team finish	Team gates ↑	Team time (s) ↓	GW / IV / S ↓
Two vehicles, homogeneous, gap 90 *s*						
	Continuous Servo	5	5/5	24.0/24	303.1 ± 3.2	0.0 / 0.0 / 0.0
	Center-then-Commit	5	5/5	24.0/24	308.0 ± 5.8	0.0 / 0.0 / 0.0
Three vehicles, heterogeneous convoy, gap 0 *s*						
	No coordination	3	3/3	36.0/36	219.7 ± 2.5	9.3 / 2.3 / 0.0
	LF(1)	3	3/3	36.0/36	262.3 ± 7.7	**0.0 / 0.0 / 0.0**
	LF(2)	3	3/3	36.0/36	288.3 ± 1.3	0.0 / 0.0 / 1.0
Three vehicles, heterogeneous convoy, gap 8 *s*						
	No coordination	3	2/3	34.3/36	254.0 ± 24.5	127.3 / 2.0 / 0.0
	LF(1)	3	**3/3**	**36.0/36**	266.0 ± 8.3	**0.0 / 0.0 / 0.0**
	LF(2)	3	3/3	36.0/36	285.7 ± 3.8	0.0 / 0.0 / 0.0

Team times are raw team elapsed mean ± sample standard deviation over full-team finishes, computed by (31); gates and events are per-condition means. *GW / IV / S*: gate and world collisions, inter-vehicle proximity events and stuck events. No run recorded an out-of-bounds event. LF(Δg) is the leader–follower policy of (28) with a margin of Δg locally estimated gates. Penalized team time differs from raw team time only where events were charged: 266.4 ± 64.1 s and 564.0 ± 463.0 s for the uncoordinated convoy at gaps 0 and 8 s, and 303.3 ± 1.3 s for LF(2) at gap 0 s, whose longer hold trips the independent stuck timer of (22). In the three-vehicle condition the leader runs Continuous Servo and both followers run Center-then-Commit; no controller or course parameter was retuned between conditions.

Table 14: Final Gen-2 27-D RecurrentPPO validation at the learning control rate. Three valid episodes were retained per official circuit; times are finished-run means and are descriptive only.

Circuit	Gates	Episodes	Finished	Mean time (s)	Collisions	OOB
Horseshoe Bay	12	3	3/3	192.5	0	0
Vertical Serpent	17	3	3/3	239.1	1	0
Mixed Endurance	22	3	3/3	365.6	0	0
All circuits	–	9	9/9	–	1	0

The selected artifact is `artifacts_gen2/ppo_27d_direct_speed_campaign_C_20260909_retry1/final_validation_50k`; the selected checkpoint contains 50,688 actual environment steps. The separate current-free rule demonstration also finishes 9/9 episodes, but its timing records are not pooled here because it is a different artifact and controller.

10.6. RQ5: Learned Controllers Under the Same Contract

Table 14 reports the final selected Gen-2 checkpoint through the same participant interface and independent referee used by the rule controllers. The policy completed all nine valid episodes: 3/3 on each circuit. There was one collision event in Vertical Serpent and no out-of-bounds events. These results are descriptive at $n = 3$ per circuit and do not support a statistical ranking.

The rule comparison is intentionally limited. A separate current-free 10-Hz reference artifact also completed 9/9 episodes with no collision, out-of-bounds or wrong-direction events. Its timing records are not pooled with the PPO timings because the campaigns have different controllers and manifests. The larger systematic rule matrix runs at 30 Hz and is reported separately; 30-Hz and 10-Hz timings are not directly comparable and are never combined into one performance estimate.

The learning result is therefore an integration and diagnosis result, not a claim that PPO outperforms the reference controller. It confirms that the Gen-2 27-dimensional local transition contract can drive a recurrent policy through the full evaluation harness, while the independent ordered-sequence referee keeps circuit completion distinct from isolated gate crossings. The official circuits were available to earlier diagnostic runs, so this is retrospective validation rather than unseen-track generalization.

11. Discussion

The experiments support a view of underwater gate racing as a benchmark for ordered-sequence autonomy rather than speed alone. A vehicle must commit to the correct target, reacquire a new target after each passage and compose those transitions over a long course. Small per-transition defects therefore amplify with course length, while a single-gate task can make a weak sequence policy look successful.

11.1. What the Evaluation Contract Reveals

The central value of Marine Race Arena is the conjunction of a fixed information boundary, an ordered referee, configurable race management and machine-readable scoring. The boundary makes controller comparisons meaningful because rule-based and learned policies receive the same information. The referee makes local progress auditable: controller estimates can be compared with privileged adjudication without allowing the latter into the control loop. The resulting residual disagreements are useful evidence, not hidden implementation details.

The configurable race manager makes this separation operational. Track geometry, participants, environmental conditions, starts, timing and penalties can be changed through scenario configuration while the participant interface and referee logic remain fixed. This permits the benchmark to vary the task and the operating condition independently of the autonomy implementation, which is important when interpreting differences between controllers.

The referee also makes failure interpretable. A missed next gate is recorded as a sequence failure rather than being blurred into a low aggregate gate count, and safety costs remain visible alongside progress. This is why the current sweep cannot be summarized by collision counts alone: a stronger disturbance can reduce collisions simply by stopping a vehicle before the contact-dense part of the course. Reporting progress, completion and events together is a benchmark requirement, not a presentation choice.

11.2. Reference Controllers, Currents and Fleets

The reference-controller comparison is a controlled ablation of visual feedback during passage. Neither controller dominates: continuous correction is useful on repeated vertical transitions, whereas suspending late visual corrections is more robust on the long mixed course and under the informative current profile. The result is evidence that the benchmark can expose a design decision that a single clean circuit would conceal, not evidence that either controller is optimal.

The fleet experiments extend the same logic from individual to team autonomy. Vehicles that are individually competent can still converge on the same aperture and collide when their interaction is not scored. A distributed leader–follower policy removes those events within the legal observation boundary, while the referee charges waiting through the stuck penalty. Thus coordination and its cost are evaluated in the same score rather than in a separate engineering trace.

11.3. Learning as a Demonstration of Generality

The final Gen-2 result supports a more precise interpretation of learning in this benchmark. A recurrent PPO controller using the 27-dimensional local transition contract completed all nine valid validation episodes, with three episodes on each official circuit. One collision event was recorded on Vertical Serpent and no out-of-bounds events were recorded. Thus, a learned controller can be integrated into the same observation, action and referee interfaces and can complete full ordered circuits under the reported current-free protocol.

This result remains an integration and diagnostic result rather than a claim that PPO is superior to the reference controllers or that it generalizes to unseen circuits. The validation uses three episodes per circuit, the official circuits had been accessible to earlier diagnostic runs, and the learned campaign uses a different control rate and provenance from the systematic rule matrix. Its timing should therefore not be ranked directly against the rule controller tables. More broadly, the experiment demonstrates that the controller-agnostic interface accommodates recurrent learning without changing race execution or adjudication, but it does not isolate the effects of the architecture, training budget, reward design or observation encoding.

The next learning experiments should extend this matched protocol to currents, multi-vehicle interaction and sealed circuits, while comparing memory, training and curriculum choices under the same independent referee. The benchmark makes these comparisons possible without changing the information boundary or the official scoring mechanism.

11.4. Implications for Benchmark Design

Taken together, the results argue for benchmarks that define both race configuration and evaluation above the simulator. Solvability, disturbance robustness, team interaction and learning generality are different questions and require separate conditions, metrics and provenance. Marine Race Arena provides a controller-agnostic interface, a configurable race manager and an independent referee for underwater ordered-gate racing; its limitations and the scope of the current evidence are given next.

12. Limitations

We state the boundaries of this work explicitly, both because they define what the reported numbers may be used for and because several of them are the natural next steps.

L1. No physical validation, and an unquantified simulation-to-reality gap. Every result in this paper was produced in simulation. No experiment was run on a physical BlueROV2 or in a water tank, and we make no claim that any reported completion rate, time or collision count would be reproduced by a real vehicle. The gap is unquantified rather than small: closing it would require a physical gate course, a calibrated acoustic positioning setup and a characterization of the optical conditions, none of which is attempted here.

L2. Backend fidelity bounds the physical claims. The benchmark inherits the hydrodynamics, rendering and sensor models of its backend. HoloOcean provides a usable and widely adopted approximation, but it is not a validated computational-fluid-dynamics model of a BlueROV2, and the current field of (9) is an analytic superposition of primitives rather than a simulated flow. Conclusions about *relative* controller behaviour are therefore better supported than conclusions about absolute achievable speeds or about the specific current magnitude at which a controller fails.

L3. Optical perception assumptions. The detector of Section 5.1 assumes gates that project to closed, near-rectangular contours of bounded aspect ratio against a background whose edge structure is less rectangular. It was developed for, and validated on, the rendering conditions of these circuits. Turbidity, marine snow, backscatter from an active light source, biofouling on the gate frames, strong caustics and low-light operation are not modelled, and the detector's geometric filters would require re-derivation under any of them.

L4. Gate appearance assumptions. All gates in this study are square frames of identical $1.5\,\mathrm{m} \times 1.5\,\mathrm{m}$ inner aperture, and the detector's size, aspect and near-field area gates are calibrated to that geometry. The known-aperture assumption is also what makes the pose extension's known-size distance estimate possible. Heterogeneous gate sizes, non-square apertures, partially occluded gates or gates that must be distinguished by appearance rather than by acoustic association are outside the current scope.

L5. Acoustic sensing is an approximation, not a channel model. The beacon model of (6) applies independent Gaussian perturbations to range, bearing and elevation with Bernoulli dropout and a range cut-off. It does not model multipath, ray bending, Doppler on the acoustic carrier, frequency-dependent absorption or the correlated errors these produce. The inter-vehicle channel of (27) is likewise an acoustic-inspired transport with range-dependent latency and independent packet loss; its loss probability, range, latency and freshness window were not independently swept, and no claim about acoustic communication performance is made.

L6. The current sweep is narrow. Quantitative current results are restricted to two named profiles on one circuit (Horseshoe Bay). The `strong` profile turned out to be a saturation condition rather than a graded difficulty, so the informative disturbance evidence rests effectively on a single profile and five seeds. A finer sweep, and the same sweep on the other two circuits, would be needed to characterize the degradation curve rather than two points on it.

L7. Limited controller diversity in the systematic matrix. The rule-based evidence covers exactly two controllers that differ in one design decision. This is deliberate — it makes the comparison a clean ablation — but the 78-run matrix has not yet been exercised against structurally different autonomy strategies, such as a model-predictive controller, a cascaded design with an inner DVL velocity loop, or a planner that reasons about several gates ahead. The separate Gen-2 recurrent-PPO validation demonstrates interface compatibility, but it does not provide a factorial comparison across controller families, circuits and disturbances. We therefore cannot claim that the difficulty ordering across the three circuits is a property of the circuits rather than of the evaluated controllers.

L8. Limited fleet scale and diversity. Fleet evidence covers two homogeneous vehicles at a 90 s gap and three heterogeneous vehicles at two gaps, on one clean circuit, with three or five seeds. Larger fleets, fleets under current or with obstacles, formation-keeping objectives, cooperative overtaking, dynamic role assignment and competitive multi-team racing are all unaddressed. The coordination result in particular rests on three seeds per condition.

L9. The pose-aware perception extension is not validated. As reported in Section 5.3, the exploratory pose front end recovers a pose in only 11.7% of frames on a dedicated oblique-view rig, with a median plane-yaw error of 48.2° and orientation-class and sign accuracies of 0.00. It is accurate in the near-frontal regime the controller actually operates in (Table 6) and unreliable outside it. It is implemented and released, but it supports no claim in this paper and should not be used as a pose estimator.

L10. The learned-control evidence is a limited validation snapshot. The final learned result is a single selected Gen-2 recurrent-PPO checkpoint using the 27-dimensional local transition contract. It was evaluated in nine valid current-free episodes, three on each official circuit, and completed all nine with one collision event. This supports the claim that a recurrent learned controller can be integrated and evaluated through the common interface; it does not establish superiority, robustness under currents, fleet-level learning, sample efficiency or transfer to unseen circuits. The validation sample is small, the checkpoint was selected from a campaign with prior diagnostic access to the official circuits, and the learning protocol is not factorially matched to the 78-run rule-controller matrix. Broader comparisons of recurrent architectures, training procedures, disturbances and sealed circuits remain open.

L11. The official circuits are no longer unseen. An exploratory holdout evaluation opened Horseshoe Bay, Vertical Serpent and Mixed Endurance, and its access ledger records all sixty accesses. They must therefore not be treated as held-out circuits for any policy trained after that point, and every learned-controller result on them in this paper is labelled a retrospective diagnostic benchmark. Future generalization claims require a sealed circuit set that has not been opened.

L12. Three circuits do not span underwater autonomy. The official track set was designed to vary length, verticality and sequencing, and it does so — 12 to 22 gates, 93.8 to 206.3 m, planar to serpentine. It nonetheless covers a narrow slice of the underwater autonomy problem. Docking, manipulation, inspection, long-horizon navigation with drift, adversarial or dynamic environments and mission-level planning are all outside the benchmark's task definition, and results here should not be read as evidence about them.

13. Reproducibility

Every number in this paper is traceable to a released artifact through a recorded chain of identifiers. This section documents that chain so a reader can verify a value, re-derive a table, or run an equivalent experiment.

13.1. Repository and Environment

The implementation, the track configurations, the controllers, the evaluation harness and the aggregated result artifacts are available in a public repository [26]. The benchmark environment is HoloOcean 2.3.0 with its Ocean world, Python 3.9.25 and the pinned dependency set in `requirements.txt`; the learning extension adds a separate pinned set in `requirements-rl.txt` (Gymnasium 1.0.0, PyTorch 2.8.0, Stable-Baselines3 2.7.1) that is deliberately installed into a distinct environment so the benchmark environment used for the frozen matrix remains unchanged. Runs reported here were executed on Windows 10 build 26200.

13.2. Configuration-Driven Experiments

A race is fully specified by one JSON file (Section 4): world bounds and environment, the ordered gate sequence with centres, passage directions and aperture sizes, the beacon network and its noise parameters, the current profiles, the obstacle generation rules, the participants with their vehicles, sensors, controllers and release delays, and the referee's validation margins, penalties and scoring rules. A single entry point runs any of them, and a controller is loaded by alias, by dotted module path or by file path plus class name without modifying the package (Section 6). Reproducing a reported condition therefore requires the track file, the controller identifier and the seed, all of which are recorded per run.

13.3. Seeds and Determinism

Beacon packet randomness is seeded by the run seed, the transmitter, the receiver and the transmission index, so reception is independent of participant count and of the order in which controllers are polled; a fleet run is reproducible for this reason. Inter-vehicle packet loss and procedural course generation draw from seeded generators. For the learning extension, seed bands are registered by role — training, validation, test and final holdout — in a seed registry committed to the repository, and each evaluation records both requested and completed seeds. The matched benchmark of Section 10.6 evaluated every controller on the same seed set and the same generated geometries, which is what makes its paired statistics valid.

13.4. Manifests, Fingerprints and Hashes

Three levels of provenance are recorded.

Per-run. Each run emits a line-delimited event log and a machine-readable summary (R5), plus metadata giving the track, controller, seed, command line, requested and actual adapter, fallback-disabled status, control timestep, requested and measured current vector, and the observation and action contract versions.

Per-experiment. The 78-run matrix carries a complete experiment manifest recording the experiment-generation commit (`df098ef4...`), a source-tree fingerprint over the package's Python and JSON files (`e7d31077...`), the HoloOcean version, the Python version, the operating system, the fallback status and the simulated current-coupling status for every run. It also asserts matrix completeness: 78 of 78 expected runs, no missing and no unexpected metadata.

Per-artifact. Learned-controller evaluations additionally record the model path and its SHA-256, the track file SHA-256 and the campaign commit. The final Gen-2 campaign records the observation contract, checkpoint selection decision, actual timestep, adapter status and per-episode result files; the selected checkpoint SHA-256 is stored beside the validation manifest.

13.5. Regenerating Tables and Figures

All result tables and figures in this paper are *post-processing* of existing artifacts: no command listed below launches the simulator or modifies a raw artifact.

```
# Result tables from the frozen 78-run matrix (also runs the penalty-identity check)
python article/regenerate_tables.py --check

# Track layouts and the controller-comparison plot, from track JSON and artifacts
python article/figures/generate_figures.py

# Journal figures: perception panels from committed artifacts
python article_journal/scripts/make_perception_figure.py

# Verify the compact immutable 78-run release package
python article_journal/scripts/package_78_matrix.py verify

# Manuscript
cd article_journal && latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

Running `regenerate_tables.py` against the frozen matrix reproduces the clean-track, current, fleet and coordination values of Tables 11, 12 and 13 exactly, and also verifies the penalty identity $T^{\text{pen}} = T^{\text{official}} + P$ on every finished run. The two journal figure scripts write a provenance JSON alongside each output recording their input files and asserting that no simulator was launched.

13.6. Re-Running Experiments

Reproducing a run rather than a table requires HoloOcean and the recorded seed. Every experimental command is stored verbatim in the per-run metadata; a clean single-vehicle run has the form

```
python -m marine_race_arena.scripts.run_marine_race \
  --track marine_race_arena/tracks/marine_race_horseshoe_bay.json \
  --benchmark-task clean_gate \
  --controller rule_gate_center_then_commit \
  --adapter holoocean --seed 0 --dt 0.033 \
  --duration 560.0 --current-profile none --official
```

and the current-free demonstration of Section 10.2 is driven by committed scripts that pass `-current-profile none` and assert `currents_actual` = $[0, 0, 0]$ in the manifest before any episode runs. Two conventions are worth restating for anyone attempting a comparison. First, the engine-free fallback adapter exists for deterministic unit testing and must be disabled for any reported result; the manifest field `adapter_actual` records which adapter actually ran. Second, the control timestep must match the condition being reproduced: $\Delta t = 0.033$ s for the 78-run matrix and $\Delta t = 0.1$ s for the current-free demonstration and final learned-controller validation. These rates define different experimental conditions; their timings must not be pooled.

13.7. Updating the Learning Results

The learning section is deliberately structured so that its validation can be refreshed without changing the benchmark protocol. A new frozen checkpoint is added by recording its SHA-256 and selection decision, running the same three seeds per official circuit, and regenerating Table 14 from the per-episode result files. The methodology of Section 9—the 27-D observation encoding, recurrent architecture, reward and evaluation separation—is independent of any one checkpoint.

14. Conclusion and Future Work

This paper presented Marine Race Arena, a configurable and reproducible benchmark and evaluation framework for onboard-only autonomous underwater gate racing. Its central contribution is a controller-agnostic observation–action interface combined with an explicit race manager and an enforced separation between autonomy and evaluation: controllers use only allow-listed onboard sensing and messages delivered through simulated channels, while an independent referee uses privileged state to adjudicate ordered crossings, penalties and completion.

The experiments show that the common contract is usable across heterogeneous circuits, currents, fleets and controller types. The reference controller completes the current-free demonstration, while the systematic matrix exposes condition-dependent controller robustness and coordination failures that clean single-vehicle scores would miss. The final Gen-2 recurrent-PPO controller also completes all nine valid current-free validation episodes through the same 27-dimensional observation and four-action interface, with one collision event and no out-of-bounds events. This validates the integration of a learned controller into the benchmark and referee, without turning the result into a claim of superiority or unseen-circuit generalization.

The evidence remains simulation-based and bounded by one backend, a narrow current sweep, limited controller diversity in the systematic matrix, a small fleet and an exploratory pose extension that is not used by the main controllers. The learned validation is based on one selected recurrent-PPO checkpoint, three episodes per circuit and circuits that are not sealed against prior diagnostic access. Future work should extend the disturbance and fleet axes, compare broader sequence-aware learning strategies under matched protocols, seal new evaluation circuits and transfer the contract to a physical BlueROV2. In each case, Marine Race Arena provides the instrument for making the comparison reproducible and meaningful.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The benchmark implementation, the track configurations, the evaluation harness and the aggregated result artifacts are publicly available at `https://github.com/AndreaBedei1/HoloDroneCompetition`. Section 13 describes the artifact layout and regeneration commands. A compact, machine-readable release of the 78-run matrix is included under `article_journal/scientific_release/matrix_78_20260715`; its deterministic checker is run with `python article_journal/scripts/package_78_matrix.py verify`. No archival DOI has been assigned yet (TODO: deposit a release snapshot and add the DOI here).

References

[1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, R. Mangharam, Autonomous vehicles on the edge: A survey on autonomous vehicle racing, IEEE Open Journal of Intelligent Transportation Systems 3 (2022) 458–488. doi:10.1109/OJITS.2022.3181510.

[2] M. O'Kelly, H. Zheng, D. Karthik, R. Mangharam, F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning, in: Proceedings of the NeurIPS 2019 Competition and Demonstration Track, Vol. 123 of Proceedings of Machine Learning Research, 2020, pp. 77–89.
URL `https://proceedings.mlr.press/v123/o-kelly20a.html`

[3] S. Hoffmann, S. Sagmeister, T. Betz, J. Bongard, S. Büttner, D. Ebner, D. Esser, G. Jank, S. Goblirsch, A. Langmann, M. Leitenstern, L. Ögretmen, P. Pitschi, A.-K. Schwehn, C. Schröder, M. Weinmann, F. Werner, B. Lohmann, J. Betz, M. Lienkamp, Head-to-head autonomous racing at the limits of handling in the A2RL challenge, arXiv preprint arXiv:2602.08571Abu Dhabi Autonomous Racing League; preprint, not peer reviewed (2026). arXiv:2602.08571.
URL `https://arxiv.org/abs/2602.08571`

[4] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, D. Scaramuzza, AlphaPilot: Autonomous drone racing, in: Robotics: Science and Systems (RSS), 2020. doi:10.15607/RSS.2020.XVI.081.
URL `https://www.roboticsproceedings.org/rss16/p081.html`

[5] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza, Champion-level drone racing using deep reinforcement learning, Nature 620 (7976) (2023) 982–987. doi:10.1038/s41586-023-06419-4.

[6] A. Manzanilla, S. Reyes, M. A. Garcia, D. A. Mercado, R. Lozano, Autonomous navigation for unmanned underwater vehicles: Real-time experiments using computer vision, IEEE Robotics and Automation Letters 4 (2) (2019) 1351–1356. doi:10.1109/LRA.2019.2895272.

[7] M. Stojanovic, J. Preisig, Underwater acoustic communication channels: Propagation models and statistical characterization, IEEE Communications Magazine 47 (1) (2009) 84–89. doi:10.1109/MCOM.2009.4752682.

[8] M. M. M. Manhães, S. A. Scherer, M. Voss, L. R. Douat, T. Rauschenbach, UUV simulator: A Gazebo-based package for underwater intervention and multi-robot simulation, in: OCEANS 2016 MTS/IEEE Monterey, 2016, pp. 1–8. doi:10.1109/OCEANS.2016.7761080.

[9] E. Potokar, S. Ashford, M. Kaess, J. G. Mangelson, HoloOcean: An underwater robotics simulator, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2022, pp. 3040–3046. doi:10.1109/ICRA46639.2022.9812353.
URL `https://www.ri.cmu.edu/publications/holoocean-an-underwater-robotics-simulator/`

[10] S. Chu, Z. Huang, Y. Li, M. Lin, D. Li, I. Carlucho, Y. R. Petillot, C. Yang, MarineGym: A high-performance reinforcement learning platform for underwater robotics, in: 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2025, pp. 17146–17153. doi:10.1109/IROS60139.2025.11246146.
URL `https://ieeexplore.ieee.org/document/11246146`

[11] M. von Benzon, F. F. Sørensen, E. Uth, J. Jouffroy, J. Liniger, S. Pedersen, An open-source benchmark simulator: Control of a BlueROV2 underwater robot, Journal of Marine Science and Engineering 10 (12) (2022) 1898. doi:10.3390/jmse10121898.

[12] P. Cieślak, Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ROS interface, in: OCEANS 2019 - Marseille, 2019, pp. 1–6. doi:10.1109/OCEANSE.2019.8867434.

[13] M. M. Zhang, W.-S. Choi, J. Herman, D. Davis, C. Vogt, M. McCarrin, Y. Vijay, D. Dutia, W. Lew, S. Peters, B. Bingham, DAVE aquatic virtual environment: Toward a general underwater robotics simulator, in: Proceedings of the IEEE/OES Autonomous Underwater Vehicles Symposium (AUV), 2022, pp. 1–8. doi:10.1109/AUV53081.2022.9965808.
URL https://arxiv.org/abs/2209.02862

[14] E. Potokar, S. Ashford, M. Kaess, J. G. Mangelson, HoloOcean: A full-featured marine robotics simulator for perception and autonomy, IEEE Journal of Oceanic Engineering 49 (4) (2024) 1322–1336. doi:10.1109/JOE.2024.3410290.

[15] A. Amer, O. Álvarez-Tuñón, H. I. Ugurlu, J. L. F. Sejersen, Y. Brodskiy, E. Kayacan, UNav-Sim: A visually realistic underwater robotics simulator and synthetic data-generation framework, in: 2023 21st International Conference on Advanced Robotics (ICAR), IEEE, 2023, pp. 570–576. doi:10.1109/ICAR58858.2023.10406819.

[16] Z. Huang, M. Buchholz, M. Grimaldi, H. Yu, I. Carlucho, Y. R. Petillot, URoBench: Comparative analyses of underwater robotics simulators from reinforcement learning perspective, in: OCEANS 2024 - Singapore, IEEE, 2024. doi:10.1109/OCEANS51537.2024.10682284.

[17] J. Tani, A. F. Daniele, G. Bernasconi, A. Camus, A. Petrov, A. Courchesne, B. Mehta, R. Suri, T. Zaluska, M. R. Walter, E. Frazzoli, L. Paull, A. Censi, Integrated benchmarking and design for reproducible and accessible evaluation of robotic agents, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020, pp. 6229–6236. doi:10.1109/IROS45743.2020.9341677.
URL https://arxiv.org/abs/2009.04362

[18] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, D. Scaramuzza, AlphaPilot: Autonomous drone racing, Autonomous Robots 46 (2022) 307–320. doi:10.1007/s10514-021-10011-y.

[19] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, in: Proceedings of the Conference on Robot Learning (CoRL), Vol. 78 of Proceedings of Machine Learning Research, 2017, pp. 1–16.
URL https://proceedings.mlr.press/v78/dosovitskiy17a.html

[20] M. Franchi, et al., Underwater robotics competitions: The european robotics league emergency robots experience with feelhippo auv, Frontiers in Robotics and AI 7 (2020) 3. doi:10.3389/frobt.2020.00003.

[21] K. Harada, R. Fukuda, Y. Mizoguchi, Y. Yamamoto, K. Mishima, Y. Tanaka, Y. Nishida, K. Ishii, Autonomous underwater vehicle with vision-based navigation system for underwater robot competition, in: Proceedings of the International Conference on Artificial Life and Robotics (ICAROB2022), Vol. 27, 2022, pp. 354–359. doi:10.5954/ICAROB.2022.OS28-2.
URL https://kyutech.repo.nii.ac.jp/record/7489/files/ICAROB2022_OS28-2.pdf

[22] M. Xanthidis, M. Kalaitzakis, N. Karapetyan, J. Johnson, N. Vitzilaios, J. M. O'Kane, I. Rekleitis, AquaVis: A perception-aware autonomous navigation framework for underwater vehicles, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021. doi:10.1109/IROS51168.2021.9636124.

[23] Y. Yang, Y. Xiao, T. Li, A survey of autonomous underwater vehicle formation: Performance, formation control, and communication capability, IEEE Communications Surveys & Tutorials 23 (2) (2021) 815–841. doi:10.1109/COMST.2021.3059998.

[24] T. Yan, Z. Xu, S. A. Gadsden, Formation control of multiple autonomous underwater vehicles: A review, Intelligence & Robotics 3 (1) (2023) 1–22. doi:10.20517/ir.2023.01.

[25] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, OpenAI gym, arXiv preprint arXiv:1606.01540 (2016). arXiv:1606.01540.
URL https://arxiv.org/abs/1606.01540

[26] A. Bedei, HoloDroneCompetition: Marine Race Arena repository, https://github.com/AndreaBedei1/HoloDroneCompetition (2026).